

Towards Geometrically Consistent Novel View Synthesis Using Gaussian Splatting

by

Rusiru Thushara

Submitted in partial fulfillment of the requirement
for the degree of
Master of Science
in
Computer Vision

Department of Computer Vision
Mohamed bin Zayed University of Artificial Intelligence
Abu Dhabi, UAE

February 2025
Copyright © 2025, Rusiru Thushara
All Rights Reserved

Examining Committee Membership

The following served on the Examining Committee for this thesis. The decision of the Examining Committee is by majority vote.

External Examiner: Hao Li
Professor, Dept. of CV,
Mohamed bin Zayed University of Artificial Intelligence

Supervisor(s): Ivan Laptev
Professor, Dept. of CV,
Mohamed bin Zayed University of Artificial Intelligence
Salman Khan
Professor , Dept. of CV,
Mohamed bin Zayed University of Artificial Intelligence
Zhiqiang Shen
Assistant Professor , Dept. of ML,
Mohamed bin Zayed University of Artificial Intelligence

Author's Declaration

I, **Rusiru Thushara**, hereby declare that the thesis entitled **Consistency-Aware Single View 3D Reconstruction Using Gaussian Splatting** is my own original work, except where specific reference is made to the contributions of others. This thesis contains material that has been published in collaboration with co-authors, as outlined below. A true and complete copy of the thesis, including any necessary revisions, has been submitted to and accepted by my examiners. Proper acknowledgment has been given throughout the thesis wherever the work of others has been referenced. I have also clearly indicated my individual contributions, as well as those made by co-authors, in the sections below.

I understand that my thesis may be made available electronically for public access.

Rusiru Thushara Kuruppu Achchige

17th March 2025

Abstract

Novel view synthesis from a single image is a challenging, yet fundamental problem in computer vision and graphics, with applications in mixed reality, gaming, and content creation. While methods such as Neural Radiance Fields (NeRF) and 3D Gaussian Splatting (3D-GS) have achieved impressive results, they often require densely captured multi-view images, limiting their practicality in real-world scenarios with sparse or single-image inputs. Recent advances in diffusion models have enabled zero-shot novel view synthesis; however, they struggle with occlusions, lack precise camera control, and often fail to maintain consistency across views.

This thesis presents a novel approach for consistency-aware single-view 3D reconstruction by leveraging the complementary strengths of video diffusion models and Gaussian splatting. Video diffusion models, trained on large-scale video datasets, excel at generating plausible motion sequences but lack inherent 3D spatial awareness, making precise viewpoint control difficult. Meanwhile, point cloud-based methods provide a coarse 3D structure but suffer from sparsity, artifacts, and inconsistencies in occluded regions. To address these challenges, we propose an integrated framework that conditions video diffusion priors on explicit 3D information derived from Gaussian splatting, enabling photorealistic rendering with smooth interpolation, view-dependent effects, and structural coherence.

By incorporating Gaussian splatting, our method achieves high-fidelity rendering with improved handling of transparency, soft edges, and occlusions compared to traditional point-cloud rendering. Through extensive experiments, we demonstrate that our approach outperforms existing single-view 3D reconstruction methods in terms of visual quality, geometric consistency, and camera pose controllability. This research contributes to the advancement of practical 3D reconstruction of single images, bridging the gap between generative models and explicit 3D representations.

Acknowledgements

I would like to express my deepest gratitude to my supervisors, **Prof. Ivan Laptev, Prof. Salman Khan, and Dr. Zhiqang Shen**, for their invaluable guidance, encouragement, and unwavering support throughout my master's journey. Their expertise and mentorship have been instrumental in shaping my research and academic growth.

I am also sincerely grateful to **Dr. Jiawang Bain** and **Dr. Fabio Pizzati** for their insightful discussions, continuous motivation, and constructive feedback. Their support has significantly enriched my research experience and contributed to my professional development.

A special thanks to my friends and colleagues at **MBZUAI** for their constant encouragement and camaraderie. Their support has been a source of strength and motivation throughout this journey.

Finally, I extend my heartfelt appreciation to everyone who has played a role in my academic and personal growth. Your support and belief in me have been truly invaluable.

*To those who dare to dream, explore, and create—
to those who inspire, challenge, and uplift—
this work is dedicated to you.*

Table of Contents

List of Tables	viii
List of Figures	ix
List of Abbreviations	xi
1 Introduction	1
1.1 Overview	1
1.2 Motivation	2
2 Background	4
2.1 3D Reconstruction and Novel View Synthesis	4
2.2 Sparse View 3D Reconstruction	9
2.3 Video Diffusion and Depth Priors for Novel View Synthesis	13
2.4 Summary	22
3 Methodology	23
3.1 Our Method	23
4 Experiments and Results	28
4.1 Experimental Settings	28
4.2 Datasets	29
4.3 Implementation Details	31
4.4 Results	31
5 Conclusion	39
5.1 Summary	39
5.2 Limitations and Future Directions	39
References	41

List of Tables

4.1	Quantitative comparison of different methods for outdoor data. The best values are highlighted in bold . Lower FVD and LPIPS values are better, while higher SSIM and PSNR values indicate improved performance.	32
4.2	Quantitative comparison of different methods for indoor data. The best values are highlighted in bold . Lower FVD and LPIPS values are better, while higher SSIM and PSNR values indicate improved performance.	32
4.3	Quantitative comparison of different methods for RealEstate-10K data. The best values are highlighted in bold . Lower FVD and LPIPS values are better, while higher SSIM and PSNR values indicate improved performance.	33

List of Figures

2.1	NeRF represents a 3D scene as a continuous volumetric function that can be optimized through differentiable rendering. Given a set of input images with known camera poses, NeRF synthesizes novel views by tracing camera rays through the scene. Specifically, points along each ray are sampled in a 5D space, comprising spatial coordinates and viewing directions (a). These points are processed by a deep multi-layer perceptron (MLP), which predicts both color and volume density at each location (b). The final image is generated using volume rendering, where accumulated colors and densities along the ray are composited to produce a realistic view (c). Since the entire rendering process is differentiable, the model is trained by minimizing the difference between the synthesized images and the ground truth observations, thereby refining the scene representation (d). Figure taken from the original paper [1].	6
2.2	3D Gaussian Splatting Pipeline: The pipeline begins with a sparse SfM point cloud, initializing a set of 3D Gaussians. These Gaussians are iteratively optimized, refining their positions, opacities, and covariance to accurately represent scene geometry and appearance. Adaptive density control adjusts their distribution, adding Gaussians in under-reconstructed areas and removing redundancies. A fast tile-based renderer enables efficient training, achieving speeds comparable to state-of-the-art radiance field methods. Once trained, the optimized Gaussians allow real-time rendering and navigation, making 3D Gaussian Splatting a compelling alternative for novel view synthesis. Figure taken from the original paper [2].	7
2.3	DepthSplat integrates depth estimation and 3D Gaussian splatting within a unified architecture, allowing for task transfer across domains. Specifically, DepthSplat features two branches: a multi-view branch for feature matching and a single-view branch for extracting monocular features. The per-view cost volumes and monocular features are merged to perform depth regression using a 2D U-Net architecture. During depth estimation, the model is trained using ground truth depth information. For Gaussian splatting, depth maps are first unprojected to 3D to serve as Gaussian centers, while an additional head predicts other Gaussian parameters. Novel views are synthesized through the splatting process. The complete model is trained with a photometric rendering loss, which can also function as an unsupervised pre-training step for the depth model. Figure taken from the original paper [3].	14
2.4	Wonderland Overview. Starting with a single image, a camera-guided video diffusion model tracks the camera’s path and creates a 3D-aware video latent. This latent is then used by a latent-based large reconstruction model to generate the 3D scene in a feed-forward manner. The video diffusion model incorporates dual-branch camera conditioning to ensure accurate pose control. Operating within the latent space, the LaLRM efficiently reconstructs high-fidelity 3D scenes across a broad scope. Figure taken from the original paper [4].	19

2.5	Overview of ViewCrafter: Given a reference image or sparse views, ViewCrafter construct a point cloud using a dense stereo model for free-view rendering. A point-conditioned video diffusion model refines the coarse renders, addressing missing regions and distortions. Figure taken from the original paper [5]. . . .	21
3.1	Overall Architecture: Given a single input image, we first duplicate it and process it through the Dust3R framework to obtain a point cloud representation. This point cloud is then used to initialize 3D Gaussians, which are subsequently optimized using a joint optimization strategy. Next, we apply differentiable rasterization to generate Gaussian renders, which are then passed into a pretrained video diffusion model for inpainting on the point cloud renders. To mitigate the distribution shift between point cloud renders and Gaussian splatting renders, we employ a neighborhood-based masking strategy before feeding the data into the diffusion model.	24
4.1	Improvement from 3DGS-Rendering Compared to PointCloud Renders; There’s a noticeable improvement within the bounding boxes shown in red.	31
4.2	Comparison of Results with and without Neighborhood based Masking; There is a clear improvement around the red bounding box showcasing the importance of the masking strategy.	32
4.3	Qualitative Comparison of Outdoor Scenes: Results shown are sampled frames from a 25-frame sequence. Frame #1 represents the input reference image, while the subsequent frames illustrate the synthesized novel views. Our method demonstrates superior rendering quality, improved view consistency, and better preservation of fine details compared to the baseline.	34
4.4	Qualitative Comparison of Indoor Scenes: Sampled frames from a 25-frame sequence are shown, with Frame #1 representing the input reference image. The subsequent frames depict the synthesized novel views, highlighting the improved rendering quality, enhanced view consistency, and better preservation of fine details achieved by our method compared to the baseline.	35
4.5	Visualization of iterative processing through the diffusion model of ViewCrafter. The first row represents the output after the first iteration, while the second row shows the result after the second iteration.	36
4.6	Results of Failure by Incooperating Depthmaps on Diffusion Generated Inconsistent Views; Results showcase that utilizing diffusion generated images for depth regression has significantly degrade the performance.	37
4.7	Intermediate supervision: At denoising process we take the output at a middle step of the process and generate gaussian splatting renders from those and pass it back to the denoising process	38
4.8	Denoising results with intermediate Gaussian splatting supervision. When intermediate supervision is conducted at later denoising steps, it leads to better results, however, pretrained diffusion model fails to remove the artifacts generated from Gaussian Splatting.	38

List of Abbreviations

AI	Artificial Intelligence.
ML	Machine Learning.
3D-GS	Three Dimensional Gaussian Splatting.
SFM	Structure from Motion.
MVS	Multi-View Stereo.
PSNR	Peak Signal-to-Noise Ratio.
SSIM	Structural Similarity Index Measure.
LPIPS	Learned Perceptual Image Patch Similarity.
FVD	Fréchet Video Distance.
FPS	Farthest Point Sampling.
MLP	Multi Layer Perceptron.
SGD	Stochastic Gradient Descent.
MDE	Monocular Depth Estimation.
CNN	Convolutional Neural Network.
GAN	Generative Adversarial Network.
RNN	Recurrent Neural Network.
LSTM	Long Short-Term Memory.
IoU	Intersection over Union.

Chapter 1

Introduction

1.1 Overview

Generating 3D scenes from sparse input views has gained popularity in recent years. This challenge comes up in a variety of applications, including autonomous navigation, virtual reality, and innovative view synthesis. However, there are many difficulties in the way of generating high-quality 3D reconstruction from just a handful of input photos. Insufficient 3D information is one of the main challenges, which makes it challenging to precisely infer the scene’s depth, occlusions, and fine features. Additionally, maintaining temporal and spatial consistency between frames along a given camera trajectory remains a crucial yet unresolved issue. Inconsistencies between generated frames can lead to noticeable artifacts such as flickering, ghosting, or misaligned structures, which degrade the overall visual experience.

In this work, we explore several strategies to address these challenges. Our primary objective is to generate high-quality, temporally consistent novel views given a single input image and a predefined camera trajectory. To achieve this, we investigate techniques that enhance depth estimation, improve scene coherence, and leverage prior knowledge about 3D structures. We experiment with various neural rendering approaches, including explicit neural representations, depth-aware synthesis, and learning-based regularization methods, to ensure smooth transitions and consistency across frames.

By systematically analyzing these techniques, we aim to improve the reliability and realism of 3D scene synthesis from sparse inputs. Our research contributes to the ongoing development of more robust and efficient methods for novel view generation, with potential applications in immersive media, robotics, and real-time rendering.

1.2 Motivation

Novel view synthesis remains a pivotal challenge in the fields of computer vision and computer graphics, underpinning advancements in immersive technologies such as video games, augmented and mixed reality, and digital content creation. While neural reconstruction techniques like NeRF [6] and 3D-Gaussian Splatting [2] have demonstrated impressive performance in generating photorealistic scenes, they typically require densely sampled multi-view imagery—an assumption that limits their deployment in scenarios where data is sparse or only a single viewpoint is available. Consequently, a more practical and demanding objective is to synthesize novel perspectives from minimal input, a task that calls for a sophisticated understanding of scene geometry, visual appearance, semantics, and occlusion handling.

In prior research, this problem has been addressed through regression-based methods [7, 8, 9, 10, 11, 12, 13, 14], which can perform well in controlled environments but often exhibit limited generalization—particularly outside specific scene categories like indoor spaces. These models are frequently constrained by their representational capacity, leading to poor performance on diverse or unseen data. More recently, diffusion-based techniques have emerged as a compelling alternative, offering zero-shot capabilities for novel view generation from a single image [15, 16, 17]. Despite their promise, such methods are often restricted to object-centric inputs and afford limited control over camera movement, typically relying on abstract pose prompts instead of explicit geometric transformations.

An alternative direction has combined depth-guided warping with diffusion-based inpainting [18, 19] to handle occlusions and enrich novel views. However, these techniques commonly struggle with maintaining consistency in occluded regions, as their inpainting is applied per frame, leading to spatial discontinuities. More recent efforts such as ViewCrafter [5] attempt to bridge this gap by conducting inpainting across multiple views. Nevertheless, the method still faces challenges with rendering fidelity, lacking smooth interpolation, realistic lighting effects, and cross-view coherence—largely due to its reliance on sparse point cloud renderings, which can introduce artifacts and degrade overall realism.

In this thesis, we address the challenge of generating photorealistic multi-view renderings of arbitrary scenes from a solitary input image, with meticulous control over camera positioning and robust consistency across synthesized perspectives. Our approach leverages the complementary strengths of video diffusion models and explicit 3D cues extracted via Gaussian splatting. Video diffusion architectures [20, 21, 22], pre-trained on extensive video corpora, are adept at modeling temporal coherence and authentic motion patterns, enabling the plausible extrapolation of dynamic content from minimal input. Nevertheless, these models inherently lack explicit three-dimensional scene comprehension, which complicates precise manipulation of virtual viewpoints. Concurrently, recent advances in dense stereo reconstruction [23, 24] have demonstrated that coarse point clouds can be rapidly inferred from sparse imagery, furnishing essential geometric priors for viewpoint regulation. However, such point cloud representations remain susceptible to artifacts, including occlusions, incomplete geometry, and inaccuracies, particularly when derived from highly limited visual evidence.

To overcome these challenges, we study a approach that integrates the generative strengths of video diffusion models with the structural guidance provided by Gaussian splatting.

Advantages of Gaussian Splatting Over Point Cloud Rendering

Gaussian splatting offers several advantages over traditional point-cloud rendering, making it a superior choice for novel view synthesis:

- **Higher Rendering Quality:** Gaussian splatting enables continuous, smooth, and realistic novel view synthesis, whereas point cloud rendering often suffers from sparsity and noticeable artifacts.
- **Improved Handling of Transparency and Soft Edges:** Unlike raw point cloud rendering, which can produce jagged edges, Gaussian splatting allows for seamless blending, effectively handling transparency and soft transitions.
- **View-Dependent Effects:** Gaussian splatting can encode view-dependent appearance, a crucial factor for achieving photorealistic rendering and capturing complex lighting effects.

This combination between Gaussian Splatting Renders and Video Diffusion Prior aims to achieve higher-fidelity novel view synthesis, ensuring smooth interpolation, photorealism, and geometric consistency across diverse scene types.

Chapter 2

Background

In this chapter, we provide an overview of key concepts and methodologies relevant to our work. We begin by discussing fundamental techniques in **3D reconstruction and novel view synthesis**, including NeRF-based methods and 3D Gaussian Splatting, which have demonstrated significant progress in generating high-quality novel views.

Next, we explore **sparse-view 3D reconstruction**, a critical area for handling limited input views, with a particular focus on Dust3R-based methods. These approaches aim to reconstruct accurate 3D structures from sparse image sets, addressing challenges such as occlusions and missing details.

Finally, we examine the role of **video diffusion models and depth priors** in novel view synthesis. Diffusion models have gained increasing attention for their ability to generate visually consistent and high-quality novel views, while depth priors contribute to improved geometric accuracy. We discuss how these techniques complement 3D reconstruction methods and their implications for our approach.

This chapter establishes the necessary foundation for understanding our proposed methodology, bridging existing techniques with our contributions.

2.1 3D Reconstruction and Novel View Synthesis

Structure-from-Motion (SfM) SfM aims to reconstruct sparse 3D structures while simultaneously estimating camera parameters from a set of images [25]. The traditional approach begins by identifying pixel correspondences across multiple images using keypoint matching techniques [26, 27, 28]. This is followed by bundle adjustment, which jointly optimizes 3D coordinates and camera parameters to refine the reconstruction. Recent advancements have integrated learning-based methods into the SfM pipeline, improving key aspects such as feature description [29], image matching [30, 31], feature refinement

[32], and bundle adjustment [33]. Despite these improvements, SfM remains a sequential process, making it susceptible to errors that propagate through its various stages.

Multi-View Stereo (MVS) MVS focuses on generating dense 3D reconstructions by triangulating surfaces from multiple viewpoints. Classical MVS assumes known camera parameters, obtained either through calibration during data acquisition [34, 35] or via SfM for in-the-wild reconstructions [25]. Existing approaches include handcrafted algorithms [36, 37], scene optimization techniques, and learning-based methods, all of which depend on accurate camera parameters. However, in real-world scenarios, inaccuracies in these parameters can degrade performance. Some recent methods aim to predict scene geometry without explicit camera parameter knowledge.

Direct RGB-to-3D Direct RGB-to-3D methods attempt to infer 3D geometry from a single RGB image. Given the inherent ambiguity of this task, these approaches leverage neural networks trained on large datasets to learn robust 3D priors. Broadly, they fall into two categories: object-centric methods that rely on class-specific shape priors and scene-centric methods that generalize to diverse environments. The latter typically build upon monocular depth estimation (MDE) networks [38], which provide depth maps that can be converted into 3D point clouds using camera intrinsics. Some approaches incorporate camera parameter estimation [39] to improve depth-based 3D reconstruction, but they remain constrained by the fundamental challenges of monocular depth estimation.

Other approaches process multiple viewpoints simultaneously to generate depthmaps specifically, pointmaps which facilitate triangulation without requiring explicit camera parameters. Previous multi-view learning approaches have attempted to replicate the SfM pipeline in a differentiable manner, but they require ground-truth intrinsics and produce depth maps alongside relative camera poses. Some architectures directly output dense pointmaps, effectively encoding 3D information while implicitly handling camera pose estimation.

Pointmaps Pointmaps offer an unconventional yet effective representation for multi-view stereo. While primarily used in visual localization [40, 41, 42] and monocular 3D reconstruction, pointmaps enable scene modeling in image space by storing 3D structure in multiple canonical views. Unlike traditional approaches that rely on explicit camera geometry, pointmaps have been used to represent dense 3D geometry, allowing for robust and efficient reconstruction in diverse scenarios.

NeRF based Methods

Deep learning has played a role in novel view synthesis from an early stage, with CNNs being used to predict blending weights or to generate texture-space representations. However, many of these approaches have a key limitation: they depend heavily on Multi-View Stereo (MVS)-based geometry, which can be restrictive. Moreover, relying on CNNs for final rendering often introduces temporal flickering, affecting the visual consistency of the generated views.

Volumetric approaches for novel view synthesis began with Soft3D [43], followed by deep learning-based methods that incorporated volumetric ray-marching [44]. These models utilized a continuous, differentiable density field to represent scene geometry. However, volumetric ray-marching is computationally expensive due to the large number of samples required for querying the volume. Neural Radiance Fields (NeRFs) [6] introduced techniques like importance sampling and positional encoding to enhance rendering quality, but their reliance on a large Multi-Layer Perceptron (MLP) significantly impacted speed.

NeRF’s success led to a wave of research aimed at improving both quality and efficiency, often by employing regularization techniques. One of the leading approaches in terms of image fidelity for novel view synthesis is Mip-NeRF360 [45], which achieves exceptional rendering quality but at the cost of long training and inference times.

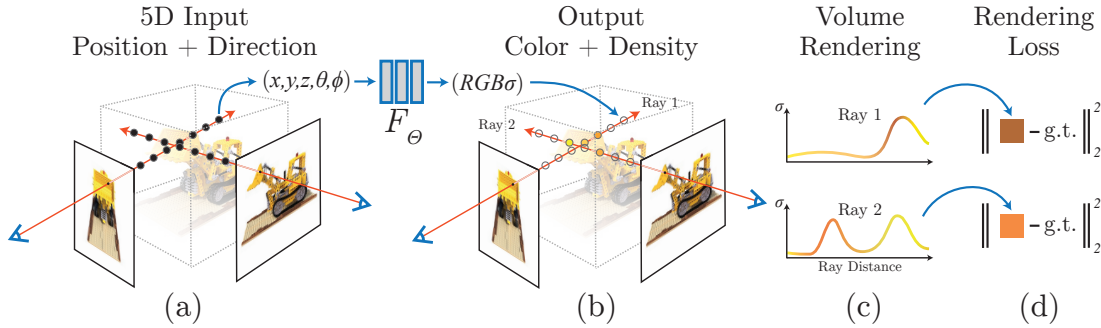


Figure 2.1: NeRF represents a 3D scene as a continuous volumetric function that can be optimized through differentiable rendering. Given a set of input images with known camera poses, NeRF synthesizes novel views by tracing camera rays through the scene. Specifically, points along each ray are sampled in a 5D space, comprising spatial coordinates and viewing directions (a). These points are processed by a deep multi-layer perceptron (MLP), which predicts both color and volume density at each location (b). The final image is generated using volume rendering, where accumulated colors and densities along the ray are composited to produce a realistic view (c). Since the entire rendering process is differentiable, the model is trained by minimizing the difference between the synthesized images and the ground truth observations, thereby refining the scene representation (d). Figure taken from the original paper [1].

Recent advancements have primarily focused on accelerating training and rendering by leveraging three key design principles: spatial data structures to store neural features that are interpolated during ray-marching, alternative encoding schemes, and modifications to MLP capacity. Several techniques have been developed around these principles, including different spatial discretization strategies [46, 47], codebook-based methods, and encoding techniques like hash tables, which allow for the use of smaller MLPs or even the complete removal of neural networks [48].

Among these approaches, two notable methods stand out: InstantNGP [49], which utilizes a hash grid and an occupancy grid to optimize computations while employing a compact MLP to model density and appearance, and Plenoxels [50], which rely on a sparse voxel grid to interpolate a continuous density field, eliminating the need for neural

networks altogether. Both techniques incorporate Spherical Harmonics—InstantNGP uses them directly for modeling directional effects, while Plenoxels leverage them to encode inputs for the color network.

While these methods achieve impressive results, they still face challenges in effectively representing empty space, depending on the scene and capture setup. Additionally, image quality is largely constrained by the structured grids used for acceleration, while rendering speed remains limited by the need to sample multiple points along each ray during ray-marching. In contrast, novel approaches like 3D Gaussian Splatting [2] overcomes these limitations by leveraging unstructured, GPU-efficient 3D Gaussians, which deliver superior rendering speed and enhanced image quality without relying on neural networks.

3D Gaussian Splatting

Recent advancements in neural scene representations have explored explicit and unstructured primitives for efficient novel view synthesis. One such approach, 3D Gaussian Splatting (3DGS), optimizes a scene representation from a sparse set of Structure-from-Motion (SfM) points, eliminating the need for normal estimation. Unlike volumetric neural representations that require expensive ray marching, 3DGS leverages *differentiable 3D Gaussians*—a compact and efficient primitive that can be directly projected onto 2D image space, enabling real-time rendering through fast α -blending [2].

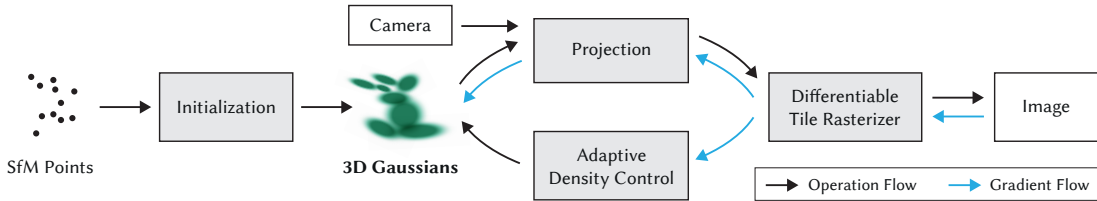


Figure 2.2: 3D Gaussian Splatting Pipeline: The pipeline begins with a sparse SfM point cloud, initializing a set of 3D Gaussians. These Gaussians are iteratively optimized, refining their positions, opacities, and covariance to accurately represent scene geometry and appearance. Adaptive density control adjusts their distribution, adding Gaussians in under-reconstructed areas and removing redundancies. A fast tile-based renderer enables efficient training, achieving speeds comparable to state-of-the-art radiance field methods. Once trained, the optimized Gaussians allow real-time rendering and navigation, making 3D Gaussian Splatting a compelling alternative for novel view synthesis. Figure taken from the original paper [2].

This method builds upon previous work in point-based rendering [51], where each point is treated as a small planar disk with an associated normal. However, estimating accurate normals from sparse SfM points is challenging, and optimizing unreliable normals often leads to instability. Instead, 3DGS models the scene using a *set of anisotropic 3D Gaussians*, parameterized by a full covariance matrix Σ defined in world space. The Gaussian function is formulated as:

$$G(x) = e^{-\frac{1}{2}x^T\Sigma^{-1}x} \quad (2.1)$$

where the covariance matrix Σ determines the shape and orientation of each Gaussian. To enable efficient projection onto 2D, the transformation from world coordinates to screen space is computed using the Jacobian of the projective transformation, allowing the Gaussians to function similarly to planar splats but without requiring explicit surface normal information.

Optimization and Adaptive Gaussian Density Control

The optimization process in 3DGS involves iteratively refining the parameters of each Gaussian—including position, color, opacity (α), and covariance—to accurately reconstruct the scene. A key challenge is ensuring that Gaussians are correctly distributed in the scene while preventing over-reconstruction. Unlike NeRF-based approaches that rely on implicit volumetric fields, 3DGS directly optimizes Gaussian parameters through *stochastic gradient descent (SGD)*, leveraging GPU-accelerated frameworks such as *PyTorch* [52]. The differentiable rasterization process is the computational bottleneck, making efficient GPU implementation crucial for real-time performance [50].

To improve representation quality, *adaptive density control* is applied. Starting from an initial sparse set of Gaussians, the model dynamically increases density in under-reconstructed regions while reducing redundancy in over-reconstructed areas. This is achieved through:

1. **Densification:** New Gaussians are introduced in areas where geometry is missing, guided by the magnitude of the view-space position gradient. If the gradient exceeds a predefined threshold, a Gaussian is duplicated and slightly displaced to better capture fine details.
2. **Splitting Large Gaussians:** In regions where a single Gaussian covers too much space, it is split into two smaller Gaussians, each scaled down by an empirically determined factor. This ensures finer scene details are accurately represented.
3. **Pruning Redundant Gaussians:** Gaussians with extremely low opacity (α) are periodically removed, preventing unnecessary computational overhead. Additionally, abnormally large Gaussians that dominate the scene are pruned to maintain an efficient representation.

Unlike *MipNeRF-360* [45] and *Plenoxels* [50], which rely on hierarchical or voxel-based representations requiring spatial compaction, 3DGS maintains an *explicit* representation of Gaussians in Euclidean space. This avoids complex warping functions or adaptive resolution schemes, allowing for *real-time* rendering without additional post-processing.

Through this optimization strategy, 3DGS effectively balances **accuracy**, **efficiency**, and **flexibility**, making it a strong alternative to traditional volumetric and neural radiance field methods for novel view synthesis.

2.2 Sparse View 3D Reconstruction

Dust3R based methods

The objective of Dust3R [23] is to develop a network capable of solving the 3D reconstruction task in the generalized stereo setting through direct regression. To achieve this, the network $\mathcal{F}$ is trained to take as input two RGB images, I^1 and I^2 , both belonging to $\mathbb{R}^{W \times H \times 3}$, and produce two corresponding pointmaps $\mathcal{X}_1^1$ and $\mathcal{X}_1^2$ within $\mathbb{R}^{W \times H \times 3}$, along with associated confidence maps $\mathcal{C}_1^1$ and $\mathcal{C}_1^2$ within $\mathbb{R}^{W \times H}$. Both pointmaps are expressed in the same coordinate frame as I^1 , a design choice that distinguishes this approach from existing methods, offering significant advantages. For simplicity, the images are assumed to have the same resolution of $W \times H$, although, in practice, their resolutions may differ.

Dust3R Network Design

The architecture of Dust3R, denoted by $\mathcal{N}$, is influenced by CroCo [53], leveraging its strong pretraining capabilities [54]. It comprises two symmetric branches—one for each input image. Each branch contains an image encoder, a transformer-based decoder, and a regression module.

Given a pair of input images J^a and J^b , both are encoded using a shared-weight Vision Transformer (ViT) [55] to produce token sequences:

$$\Phi^a = \text{Encoder}(J^a), \quad \Phi^b = \text{Encoder}(J^b).$$

These token embeddings are processed through a multi-layer decoder which jointly reasons over both views. Each decoder block applies self-attention (intra-view), cross-attention (inter-view), and a feedforward layer. The tokens are updated at layer t as:

$$\Gamma_t^a = \text{Decoder}_t(\Gamma_{t-1}^a, \Gamma_{t-1}^b), \quad \Gamma_t^b = \text{Decoder}_t(\Gamma_{t-1}^b, \Gamma_{t-1}^a),$$

where $t = 1, \dots, T$, and the initialization is $\Gamma_0^a = \Phi^a$, $\Gamma_0^b = \Phi^b$.

Finally, each branch passes its tokens through a regression head to produce a 3D point map and a confidence map:

$$\mathbf{P}^a, \mathbf{Q}^a = \text{Head}(\Gamma_0^a, \dots, \Gamma_T^a), \quad \mathbf{P}^b, \mathbf{Q}^b = \text{Head}(\Gamma_0^b, \dots, \Gamma_T^b).$$

The predicted point maps $\mathbf{P}^a$ and $\mathbf{P}^b$ are up to an unknown scale. Importantly, the network is not constrained by any geometric priors, allowing flexibility but potentially deviating from physical camera models. Despite this, it learns geometry-consistent mappings solely from data, and the architecture accommodates powerful pretrained representations better than many hand-crafted designs.

Training Objective

Dust3R is trained with two main loss components: a 3D point regression loss and a confidence-aware weighting term.

3D Point Regression Loss

The main training signal compares the predicted 3D coordinates to ground-truth annotations. For valid pixel sets $\Omega^a, \Omega^b \subseteq \{1, \dots, W\} \times \{1, \dots, H\}$, the loss for pixel u in view $v \in \{a, b\}$ is:

$$\mathcal{L}_{3D}(v, u) = \left\| \frac{1}{s} \mathbf{P}^v(u) - \frac{1}{\hat{s}} \hat{\mathbf{P}}^v(u) \right\|_2.$$

To address the scale ambiguity, both predictions and ground truth are normalized by their average distance to the origin:

$$\mathcal{S}(\mathbf{P}^a, \mathbf{P}^b) = \frac{1}{|\Omega^a| + |\Omega^b|} \sum_{v \in \{a, b\}} \sum_{u \in \Omega^v} \|\mathbf{P}^v(u)\|_2.$$

Confidence-Aware Loss

Some 3D points, such as those on reflective or textureless surfaces, are inherently uncertain. To model this, the network also predicts a per-pixel confidence score. The full loss function incorporates these scores:

$$\mathcal{L}_{\text{conf}} = \sum_{v \in \{a, b\}} \sum_{u \in \Omega^v} \mathbf{Q}^v(u) \cdot \mathcal{L}_{3D}(v, u) - \lambda \log \mathbf{Q}^v(u),$$

where λ is a regularization coefficient. The confidence scores are constrained to be positive by setting:

$$\mathbf{Q}^v(u) = 1 + \exp(\tilde{\mathbf{Q}}^v(u)) > 1.$$

This setup incentivizes the model to express uncertainty in hard-to-predict regions while still learning reliable predictions in confident areas—without any need for explicit confidence supervision.

Applications and Extensions

Dust3R’s architecture has been widely adopted in follow-up work. For instance, Mast3R [?] enhances Dust3R with a specialized matching module to improve correspondence accuracy using the regressed point maps. While Dust3R performs well under significant viewpoint changes, its lack of dedicated matching supervision can limit correspondence precision. Mast3R addresses this by introducing reciprocal matching mechanisms in a contrastively trained feature space, offering finer control over spatial consistency.

Matching Prediction and Improvements in Mast3R

Matching Prediction Head and Loss: To overcome Dust3R’s limitations in precise correspondence estimation, Mast3R introduces an additional head that predicts dense local descriptors for both images. This head generates two descriptor maps:

$$\begin{aligned} D^1 &= \text{Head}_{\text{desc}}^1([H^1, H'^1]), \\ D^2 &= \text{Head}_{\text{desc}}^2([H^2, H'^2]), \end{aligned} \tag{2.2}$$

where $[H^v, H'^v]$ denotes the concatenation of decoder outputs from different stages of the network for image $v \in \{1, 2\}$. Each head is a two-layer MLP with GELU activation, and the resulting descriptors are ℓ_2 -normalized to unit length.

For training, Mast3R employs the InfoNCE loss [56], a contrastive learning objective that encourages descriptors to be discriminative and match only their true correspondences. Unlike regression losses that tolerate approximate matches, InfoNCE frames matching as a classification task, penalizing deviations from exact pixel-level alignment. The overall training objective combines the confidence-aware regression loss and the descriptor matching loss:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{conf}} + \beta \cdot \mathcal{L}_{\text{match}}, \quad (2.3)$$

where β is a weighting factor balancing the two components.

Fast Reciprocal Matching: To efficiently extract high-confidence correspondences, Mast3R adopts a reciprocal matching scheme. A naive approach would compute pairwise similarity scores for all pixel pairs, leading to $O(W^2H^2)$ complexity. Although optimized nearest-neighbor algorithms exist, their performance degrades in high-dimensional feature spaces and cannot match Mast3R’s runtime efficiency.

Instead, Mast3R initiates matching from a sparse set of sampled pixels in the first image. Each sampled pixel is mapped to its nearest neighbor in the second image using the descriptor similarity, and then mapped back to the first image. If the original pixel is recovered in this backward step, it is retained as a reciprocal match. This process is repeated iteratively, with converged correspondences filtered out at each step. The strategy dramatically reduces computational overhead while preserving high-quality matches. Moreover, the reciprocal check inherently filters out outliers, improving robustness.

Coarse-to-Fine Matching: Due to the quadratic complexity of attention layers, Mast3R processes images with a maximum resolution of 512 pixels along the longer dimension. This constraint necessitates downscaling high-resolution inputs, potentially discarding fine details important for precise localization and reconstruction.

To address this, Mast3R implements a coarse-to-fine refinement mechanism. Initial correspondences are established on the low-resolution image pair. Then, high-resolution crops—each limited to a 512-pixel span—are extracted in overlapping windows. Matching is performed within these local crops, and results are projected back to the original image scale. This hierarchical refinement enables the recovery of fine-grained correspondences while maintaining tractable inference time.

Through these architectural and algorithmic enhancements, Mast3R significantly improves upon Dust3R’s framework—achieving higher accuracy in dense matching, greater robustness to outliers, and improved efficiency in both training and inference.

InstantSplat [57] is also a fast and efficient system for reconstructing large-scale 3D representations from sparse-view images. Unlike traditional Structure-from-Motion (SfM) methods, it uses an end-to-end optimization framework with geometric priors, requiring as few as 2–3 images for high-quality reconstruction. InstantSplat introduces Gaussian Bundle Adjustment to link 2D and 3D, optimizing both scene representation and camera parameters via gradient-based photometric error. It also incorporates co-visibility expansion for

dense surface initialization. Achieving optimization in just 7.5 seconds, InstantSplat significantly improves reconstruction quality and generalizes well to diverse 3D representations, making it compatible with other neural reconstruction methods.

Gaussian Bundle Adjustment in InstantSplat Given multiple views and an initial 3D representation defined by a set of Gaussians $\mathbf{G}$ and camera poses $\mathbf{T}$, a differentiable rasterizer from Gaussian Splatting is employed to render images corresponding to the target poses. The optimization process follows a gradient descent approach to minimize photometric error, reducing the disparity between the 3D representation and observed 2D images. This iterative refinement updates $\mathbf{G}$ and $\mathbf{T}$ to better align with representative orientations:

$$\mathbf{G}^*, \mathbf{T}^* =_{\mathbf{G}, \mathbf{T}} \sum_{v \in N} \sum_{i=1}^{HW} \left\| \tilde{\mathbf{C}}_v^i(\mathbf{G}, \mathbf{T}) - \mathbf{C}_v^i(\mathbf{G}, \mathbf{T}) \right\|. \quad (2.4)$$

Here, $\mathbf{C}$ represents the rasterization function applied to $\mathbf{G}$ at $\mathbf{T}$, while $\tilde{\mathbf{C}}$ corresponds to the observed 2D images. The initialization of $\mathbf{G}$ and $\mathbf{T}$ is based on pairwise predictions derived from MAST3R, with further adjustments in a post-processing step.

Aligning Camera Positions for Test Views A pose-free scenario necessitates extra optimization to infer test view poses, in contrast to traditional datasets where camera poses are pre-calculated with complete access to all training and testing views. The taught 3D model stays stable while the test view poses are optimized, using a technique similar to INeRF. In order to provide alignment for precise visual quality evaluations and posture estimation, this procedure attempts to lessen photometric differences between the synthetic and real images.

Scaling Global Alignment from Pairwise Predictions In addition to camera intrinsics and extrinsics, stereo point maps are predicted using the MAST3R stereo model. To achieve a globally aligned reconstruction, a connectivity graph $\mathcal{G}(\mathcal{V}, \mathcal{E})$ is constructed, where vertices $\mathcal{V}$ represent input views, and edges $e = (n, m) \in \mathcal{E}$ indicate shared visual information between images. The initially predicted point maps $\{(\mathbf{P}_i \in \mathbb{R}^{H \times W \times 3})\}_{i=1}^N$ are adjusted into a globally consistent coordinate system by refining transformations and scale factors. However, inconsistencies in estimated focal lengths persist after optimization. Assuming a single-camera capture setup, the focal length estimates are stabilized by averaging across all training views, leading to improved visual quality.

Reducing Redundancy in Co-Visible Regions Using point maps from all training images often introduces redundant representations due to overlapping regions, resulting in duplicated pixel-aligned point clouds. This redundancy inflates the number of free parameters and slows optimization. To mitigate this, co-visibility analysis is applied to eliminate duplicate pixels.

View Ranking is performed by computing confidence scores from the confidence maps of each image. Higher scores indicate more reliable initial geometry, and points from lower-

confidence views are pruned to enhance overall point quality:

$$v_i = \frac{1}{|\mathbf{O}_i|} \sum_{o \in \mathbf{O}_i} o, \quad (2.5)$$

where v_i represents the average confidence score of view i .

Cross-View Consistency Checking begins with the view that has the lowest confidence. Point maps from higher-confidence views are projected onto the low-confidence view, and depth consistency is evaluated. Points with large depth discrepancies are pruned using the criterion:

$$\mathbf{D}_{\text{proj},j} = \bigcap_{\{i|v_i > v_j\}} \text{Proj}_j(\tilde{\mathbf{P}}_i), \quad (2.6)$$

$$\mathcal{M}_j = \begin{cases} 1 & \text{if } |\mathbf{D}_{\text{proj},j} - \mathbf{D}_{\text{orig},j}| < \theta, \\ 0 & \text{otherwise} \end{cases}, \quad (2.7)$$

$$\mathbf{P}_j = (1 - \mathcal{M}_j) \cdot \tilde{\mathbf{P}}_j. \quad (2.8)$$

Here, $\mathcal{M}_j$ denotes the visibility mask, which filters out redundant points based on a pre-defined depth threshold θ . This process iterates across all views in ascending confidence order, ensuring a refined and compact representation of the scene.

2.3 Video Diffusion and Depth Priors for Novel View Synthesis

Depth Priors for Novel View Synthesis

Multi-View Depth Estimation. Multi-view depth estimation is an essential part of classical multi-view stereo pipelines. It leverages photometric consistency across multiple images to perform feature matching and generate depth maps for a reference image. Advances in learning-based approaches [58] have significantly enhanced depth estimation accuracy. Despite these improvements, learning-based methods still face challenges in scenarios where the assumption of photometric consistency does not hold, such as occlusions, low-textured surfaces, and reflective materials.

Monocular Depth Estimation. Recent developments in monocular depth estimation [59, 60, 61] have demonstrated remarkable precision in diverse real-world datasets. However, scale ambiguity remains a fundamental limitation, restricting its direct use in applications such as 3D reconstruction [39]. To address this, features from pre-trained monocular depth models [62] are incorporated to enhance feature-matching-based multi-view depth estimation. This combination maintains multi-view consistency while improving robustness in complex environments, including regions with minimal texture and reflective surfaces.

Feed-Forward Gaussian Splatting. Several feed-forward 3D Gaussian splatting techniques [63, 64, 65, 66] have been introduced due to their efficiency and ability to

manage sparse view settings. Methods such as pixelSplat [63] and Splatter Image [67] predict 3D Gaussians from image features, while MVSplat [64] improves geometric accuracy by incorporating feature matching through cost volumes. However, these methods struggle in regions with uniform texture or high reflectivity. To mitigate this, integrating monocular depth features from pre-trained models [62] enhances depth prediction and 3D Gaussian reconstruction.

Depth and Gaussian Splatting. Flash3D [68] employs a pre-trained monocular depth model to generate 3D Gaussian representations from single images. However, the focus here is on large-scale multi-view scene reconstruction. TranSplat [69] integrates monocular depth for Gaussian reconstruction, but its approach refines coarse depth predictions derived from cost volumes, making it susceptible to error propagation. The proposed approach combines cost volume and monocular features at an earlier stage, reducing this issue and improving performance beyond TranSplat. Additional research [70] incorporates depth loss in Gaussian splatting optimization, with feed-forward and per-scene optimization strategies being complementary techniques that can be combined for further enhancement.

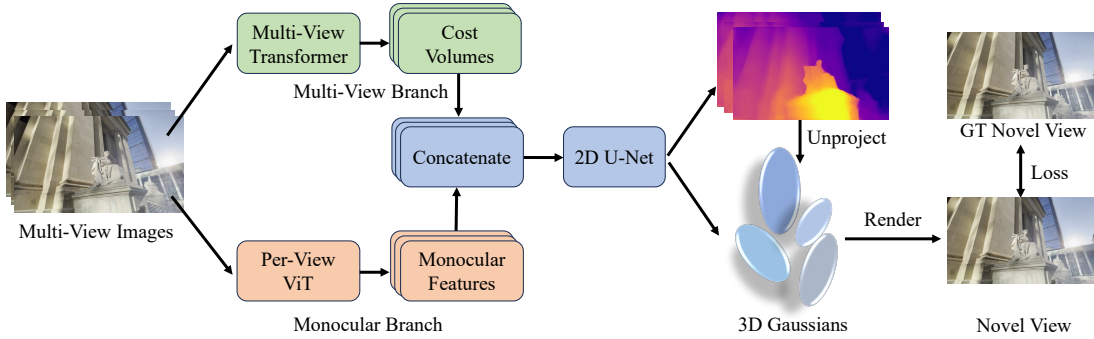


Figure 2.3: DepthSplat integrates depth estimation and 3D Gaussian splatting within a unified architecture, allowing for task transfer across domains. Specifically, DepthSplat features two branches: a multi-view branch for feature matching and a single-view branch for extracting monocular features. The per-view cost volumes and monocular features are merged to perform depth regression using a 2D U-Net architecture. During depth estimation, the model is trained using ground truth depth information. For Gaussian splatting, depth maps are first unprojected to 3D to serve as Gaussian centers, while an additional head predicts other Gaussian parameters. Novel views are synthesized through the splatting process. The complete model is trained with a photometric rendering loss, which can also function as an unsupervised pre-training step for the depth model. Figure taken from the original paper [3].

DepthSplat [3] bridges the gap between Gaussian splatting and depth estimation, two areas that are often explored separately. It introduces a robust multi-view depth model by utilizing pre-trained monocular depth features, enabling high-quality 3D Gaussian splatting reconstructions in a feed-forward manner. Additionally, DepthSplat demonstrates that Gaussian splatting can act as an unsupervised pre-training objective, helping to develop

more powerful depth models using large-scale unlabeled datasets. Through extensive ablation studies and cross-task transfer experiments, DepthSplat highlights the strong synergy between these two domains, showing how they can mutually enhance each other.

Given a set of N input images $\{I^i\}_{i=1}^N$ with dimensions $H \times W \times 3$, along with corresponding projection matrices $\{\mathbf{P}_i\}_{i=1}^N$, the objective is to estimate dense per-pixel depth maps $\mathbf{D}_i \in \mathbb{R}^{H \times W}$ and the per-pixel Gaussian parameters $\{(\mu_j, \alpha_j, \Sigma_j, c_j)\}_{j=1}^{H \times W \times N}$. The core pipeline consists of multi-view feature matching, monocular depth feature extraction, depth regression, and Gaussian parameter estimation.

Multi-View Feature Matching in DepthSplat Multi-view features are extracted using a Swin Transformer-based architecture, followed by constructing cost volumes that encode feature matching across different views. The feature extraction process utilizes a weight-sharing ResNet backbone to generate downsampled representations, which are then refined using cross-view self-attention mechanisms. Cost volumes are computed via a plane-sweep approach, warping features from neighboring views and measuring feature similarity using dot-product operations. This enables accurate depth estimation while maintaining computational efficiency.

Monocular Depth Feature Integration in DepthSplat To address challenges such as occlusions and low-texture regions, monocular depth priors are incorporated using a pre-trained ViT-based depth estimator. Monocular depth features are bilinearly upsampled to match the cost volume resolution and concatenated along the channel dimension. This integration aids in refining depth predictions where multi-view matching alone is insufficient.

Depth Regression in DepthSplat Depth regression is performed using a 2D U-Net, which processes concatenated cost volumes and monocular features. A hierarchical refinement strategy further improves depth accuracy by leveraging high-resolution feature maps in a multi-scale architecture. The final depth predictions are obtained via a softmax-based depth selection mechanism, ensuring robust and consistent depth estimation.

Gaussian Parameter Prediction in DepthSplat The estimated depth maps are unprojected into 3D using camera parameters to obtain Gaussian centers μ_j . A lightweight U-Net is then employed to predict the remaining Gaussian parameters, including opacity (α_j), covariance (Σ_j), and color (c_j). These parameters enable rendering high-fidelity novel views via Gaussian splatting.

Training Loss in DepthSplat consists of two parts, i.e. Depth Estimation Loss and View Synthesis Loss.

Depth Estimation Loss The depth estimation network is trained using an ℓ_1 loss combined with gradient losses to enforce smoothness:

$$L_{\text{depth}} = \alpha \cdot |D_{\text{pred}} - D_{\text{gt}}| + \beta \cdot |\partial_x D_{\text{pred}} - \partial_x D_{\text{gt}}| \quad (2.9)$$

$$+ \beta \cdot |\partial_y D_{\text{pred}} - \partial_y D_{\text{gt}}|, \quad (2.10)$$

where $\alpha = 20$ and $\beta = 20$.

View Synthesis Loss For novel view synthesis, the Gaussian splatting model is trained using a combination of mean squared error (MSE) and LPIPS losses:

$$L_{\text{gs}} = \sum_{m=1}^M \left(\text{MSE}(I_{\text{render}}^m, I_{\text{gt}}^m) + \lambda \cdot \text{LPIPS}(I_{\text{render}}^m, I_{\text{gt}}^m) \right), \quad (2.11)$$

where M is the number of novel views and $\lambda = 0.05$.

Video Diffusion Priors for Novel View Synthesis

Latent Video Diffusion Transformers. Recent advancements in video diffusion models have focused on latent space representations for improved efficiency [71, 72] while utilizing transformer-based architectures to enhance scalability and quality [73]. Specifically, given an input video $x \in \mathbb{R}^{T \times H \times W \times 3}$, where $H \times W$ represents the spatial dimensions and T is the number of frames, a 3D-VAE encoder $\mathcal{E}$ [71] compresses the video into a latent representation $z \in \mathbb{R}^{t \times h \times w \times c}$, where t , h , w , and c correspond to the temporal length, height, width, and channel dimensions. The compression rates in both spatial and temporal domains are defined as $r_s = \frac{H}{h} = \frac{W}{w}$ and $r_t = \frac{T}{t}$, respectively. During training, a noisy version of the latent is generated via the forward diffusion process as $z_\tau = \alpha_\tau z + \sigma_\tau \epsilon$, where $\epsilon \sim \mathcal{N}(0, I)$, and α and σ are determined by the noise scheduler, parameterized by the diffusion time τ . This noisy latent z_τ is then passed through a transformer model D_θ , parameterized by θ , which first transforms z_τ into visual tokens by patchifying it into a long sequence $o_v \in \mathbb{R}^{N_v \times d_v}$, where N_v is the sequence length and d_v is the token dimensionality. The sequence o_v is subsequently processed by multiple transformer blocks. The final output of the transformer is unpatchified and mapped back to the original latent space, where the loss is computed. The loss function is defined as $\mathcal{L}_\theta \triangleq \mathbb{E}_{x \sim p_{\text{data}}, \tau \sim p_\tau} [\|D_\theta(z_\tau; y, \tau) - \epsilon\|_2^2]$. Here, y represents the conditional signal. The exact formulation of the objective may differ depending on the model’s parameterization [72].

Camera Guided Latent Video Diffusion

Camera-Conditioned Video Diffusion Models have gained significant attention in recent research [72, 74, 75]. Earlier works focused on grouping camera movements [72] and utilizing LoRA [76] modules within video diffusion models to control specific camera actions. Subsequent studies have explored injecting camera matrices directly into video diffusion models [75, 77], though these methods sometimes result in a decrease in the visual quality of generated outputs. VD3D [74] employs a controlnet-like conditioning mechanism with cross-attention to manage camera control. However, due to the high computational demands, they limit pose control to a low-resolution video generator within a cascaded pipeline, which leads to less precise pose control and reduced visual fidelity.

MVSplat also utilizes diffusion to generate novel views. They start with K sparse-view images $\mathcal{I} = \{I^i\}_{i=1}^K$, where each $I^i \in \mathbb{R}^{H \times W \times 3}$, along with their respective camera projection matrices $\mathcal{P} = \{P^i\}_{i=1}^K$. Each camera projection matrix P^i is determined by the intrinsic matrix $\mathbf{K}^i$, rotation matrix $\mathbf{R}^i$, and translation vector $\mathbf{t}^i$, and is represented

as $P^i = \mathbf{K}^i[\mathbf{R}^i|\mathbf{t}^i]$. The aim is to learn a mapping f_θ that converts the images into 3D Gaussian parameters:

$$f_\theta : \{(I^i, P^i)\}_{i=1}^K \mapsto \{(\mu_j, \alpha_j, \Sigma_j, c_j)\}_{j=1}^{H \times W \times K},$$

where f_θ is a feed-forward network with parameters θ , which are learned through optimization on a large-scale training set. The network predicts 3D Gaussian parameters—such as position μ_j , opacity α_j , covariance Σ_j , and color c_j (modeled via spherical harmonics)—in a pixel-aligned manner. This results in $H \times W \times K$ total 3D Gaussians for K input images, each of size $H \times W$.

In MVSplat, the depth estimation framework is designed using 2D convolutional layers along with attention-based mechanisms. This design choice diverges from traditional Multi-View Stereo (MVS) methods that often rely on 3D convolutions, such as those found in MVSNet [58] and MVSNeRF [78]. By avoiding 3D operations, the model reduces computational demands. The system is organized into several main modules: feature extraction from multiple views, assembling and refining the cost volume, and subsequent stages for depth prediction and enhancement.

Multiview Feature Extraction in MVSplat. To create the cost volumes, MVSplat begins by extracting features from each input image using a hybrid of convolutional and transformer-based components. Initially, a lightweight CNN inspired by ResNet processes each view to produce downsampled feature maps. These features are then fed into a multi-view transformer module that employs both self-attention to capture intra-view patterns and cross-attention to integrate information across views. For efficient processing, it incorporates Swin Transformer-style localized attention [79]. When multiple input views are provided, cross-attention operations are applied for each view in relation to the others, using shared learnable parameters. The result is a set of cross-view aware features $\{F^i\}_{i=1}^K$, where each feature map $F^i \in \mathbb{R}^{\frac{H}{4} \times \frac{W}{4} \times C}$ captures both local and contextual information from multiple perspectives.

Cost Volume Construction in MVSplat. A central component of the MVSplat architecture is the cost volume, which captures the similarity of features across multiple views at various depth hypotheses, leveraging a plane-sweep strategy [58]. For K input views, the model constructs K individual cost volumes, one for each reference view, to estimate their corresponding depth maps.

Consider building the cost volume for a particular view i . Given predefined near and far depth bounds, we uniformly sample D candidate depths $\{d_m\}_{m=1}^D$ in inverse depth space to ensure finer sampling of closer depths. For each candidate depth d_m , we project the feature map F^j from another view j into the coordinate space of view i using their respective camera projection matrices P^j and P^i , along with d_m . This results in a set of warped features:

$$F_{d_m}^{j \rightarrow i} = \mathcal{W}(F^j, P^i, P^j, d_m) \in \mathbb{R}^{\frac{H}{4} \times \frac{W}{4} \times C}, \quad m = 1, 2, \dots, D,$$

where $\mathcal{W}$ represents the differentiable warping operation [80]. To assess the similarity between the reference view and the warped view, a normalized dot product is computed between F^i and each $F_{d_m}^{j \rightarrow i}$, producing a correlation map:

$$C_{d_m}^i = \frac{F^i \cdot F_{d_m}^{j \rightarrow i}}{\sqrt{C}} \in \mathbb{R}^{\frac{H}{4} \times \frac{W}{4}}, \quad m = 1, 2, \dots, D.$$

For scenes with more than two views, the same process is repeated across all other views $j \neq i$, and their respective correlation maps are averaged at each pixel location. This aggregation allows the model to flexibly handle multiple input views.

The final cost volume for reference view i is assembled by stacking the D correlation maps along the depth dimension:

$$C^i = [C_{d_1}^i, C_{d_2}^i, \dots, C_{d_D}^i] \in \mathbb{R}^{\frac{H}{4} \times \frac{W}{4} \times D}.$$

By repeating this procedure for all K input views, the model constructs a set of cost volumes $\{C^i\}_{i=1}^K$, one for each view.

Cost Volume Refinement. To address the ambiguity of cost volumes in regions lacking texture, MVSpLat applies a refinement stage utilizing a compact 2D U-Net [81]. This U-Net receives a concatenation of the Transformer-derived features F^i and the initial cost volume C^i , and predicts a residual volume $\Delta C^i \in \mathbb{R}^{\frac{H}{4} \times \frac{W}{4} \times D}$. This residual is added to the original cost volume to yield the refined version:

$$\tilde{C}^i = C^i + \Delta C^i \in \mathbb{R}^{\frac{H}{4} \times \frac{W}{4} \times D}.$$

To facilitate information sharing across different viewpoints, the U-Net includes three cross-view attention modules at its coarsest resolution level. These modules operate by computing interactions between a given view and all others, enabling the model to scale effectively with a variable number of input views. Once refined, the low-resolution cost volume $\tilde{C}^i$ is upsampled using a CNN to generate the full-resolution volume $\hat{C}^i \in \mathbb{R}^{H \times W \times D}$.

Depth Estimation. For each view, depth prediction is obtained by applying a softmax operation along the depth dimension of the refined cost volume. This produces a probability distribution over the candidate depths $G = [d_1, d_2, \dots, d_D] \in \mathbb{R}^D$. The final depth V^i for view i is computed as a weighted sum:

$$V^i = \text{softmax}(\hat{C}^i) G \in \mathbb{R}^{H \times W}.$$

Depth Refinement. To further improve the quality of the predicted depth maps, a second refinement stage is introduced. This module also uses a lightweight 2D U-Net that takes as input the multi-view images, feature maps, and the initial depth estimates. It outputs residual corrections to the depth predictions, which are added to obtain the final depth outputs. Analogous to the cost volume refinement process, this U-Net includes cross-view attention layers at its lowest resolution to allow depth information to be shared across views.

Wonderland [4] also achieves remarkable advancements in single-view conditioned 3D scene generation, offering notable improvements in pose control, novel view synthesis, and

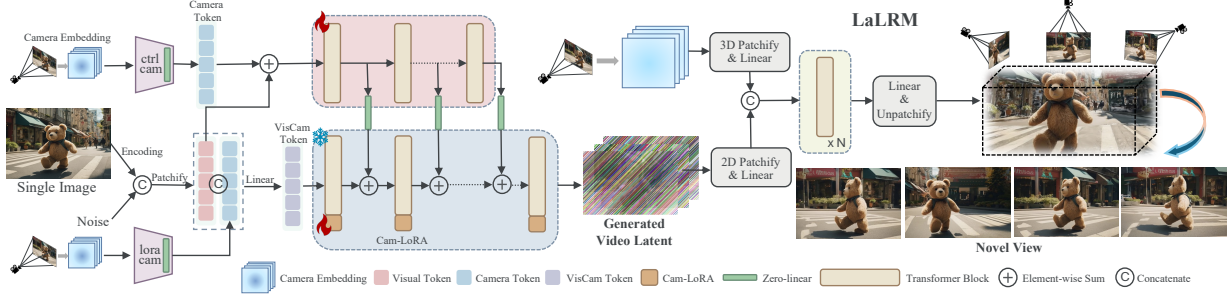


Figure 2.4: Wonderland Overview. Starting with a single image, a camera-guided video diffusion model tracks the camera’s path and creates a 3D-aware video latent. This latent is then used by a latent-based large reconstruction model to generate the 3D scene in a feed-forward manner. The video diffusion model incorporates dual-branch camera conditioning to ensure accurate pose control. Operating within the latent space, the LaLRM efficiently reconstructs high-fidelity 3D scenes across a broad scope. Figure taken from the original paper [4].

the ability to handle diverse scenes. Its generative approach and innovative conditioning strategy make it a significant contribution to the field.

One of the primary innovations in Wonderland is the use of a dual-branch camera conditioning strategy. This allows the model to generate 3D-consistent multi-view representations of scenes, offering more precise pose control than existing methods.

Furthermore, Wonderland’s reconstruction pipeline operates in the latent space, leveraging the generative capabilities of a video diffusion model. This allows it to produce high-quality, diverse images and generate wider views, going beyond traditional object-level reconstruction. The model’s ability to handle more complex, generic, and out-of-domain scenes further underscores its versatility and potential in 3D scene generation.

Wonderland comprises of two major components; i.e Camera-Guided Video Latent Generation and Latent Large Reconstruction Model.

Camera-Guided Video Latent Generation Video diffusion models have shown promise in generating high-quality visualizations, but they struggle with synthesizing 3D-aware latents due to the lack of explicit control over pose trajectory. This challenge is addressed by incorporating camera-guided video generation techniques that enable controllable and comprehensive exploration of static scenes, thereby generating latents suitable for 3D reconstruction.

Camera Representation To achieve precise control over the camera pose, the authors propose enriching the condition information by using pixel-level positional representations rather than relying on frame-level camera parameters. This is done using the Plücker embedding, which captures the camera’s rotation, translation, and intrinsic parameters. The Plücker coordinate for a pixel at position (u_f, v_f) in frame f is formulated as:

$$\dot{p}_{u_f, v_f} = (\mathbf{t}_f \times d'_{u_f, v_f}, d'_{u_f, v_f}) \in \mathbb{R}^6$$

where $\times$ denotes the cross-product, and $d'_{(\cdot)} = \frac{d_{(\cdot)}}{\|d_{(\cdot)}\|}$ is the normalized ray direction. This approach results in a Plücker embedding $p \in \mathbb{R}^{T \times H \times W \times 6}$, which encodes spatial-temporal camera pose information for the entire video.

Dual Branch Camera Guidance The authors propose a dual-branch conditioning mechanism to enable static scene generation while maintaining high visual quality. Directly adapting pretrained video diffusion transformers to produce static scenes with precise camera control is challenging due to the entanglement of spatial-temporal attention operations. Minor changes to the architecture can degrade visual quality, and fine-tuning on small datasets without architectural modifications can lead to overfitting.

To address these challenges, the authors introduce a dual-branch conditioning mechanism inspired by ControlNet and LoRA. The first branch, based on ControlNet, integrates camera information through a trainable copy of the base transformer blocks, adding the camera tokens element-wise to the video tokens. This enables the model to control the camera trajectory effectively while maintaining the quality of the generated visuals.

The second branch, incorporating LoRA, uses a channel-wise concatenation of the camera and video tokens followed by a tailored linear layer. This approach minimizes computational cost while enhancing camera control and enabling the model to adapt better to the static nature of the training set. By injecting camera information into the main branch and using LoRA for fine-tuning, the authors ensure that the pretrained video transformer can be adapted with minimal changes, improving the overall static nature of the generated scenes.

In conclusion, this work proposes a novel method for generating 3D-aware latents using video diffusion models with precise camera control. The dual-branch conditioning mechanism effectively integrates camera information while preserving visual quality, making the model suitable for downstream 3D reconstruction tasks.

Latent Large Reconstruction Model The authors also introduce the Latent Large Reconstruction Model (LaLRM) as a novel approach for 3D reconstruction. Given a single image, their camera-guided video model generates compact video latents that capture consistent multi-view appearance and geometric information of a 3D scene. These video latents are then elevated to explicit 3D representations (3DGS) through LaLRM, significantly reducing memory and time costs compared to previous methods that optimize on an image-level per-scene basis.

Unlike earlier feed-forward 3D reconstruction techniques operating in image space, LaLRM works within the latent space, offering several advantages. First, the video latents provide diverse and high-fidelity visual details from the video frame sequence. Second, the latent space is highly compressed, allowing the model to handle large-scale scenes efficiently in terms of memory. Finally, the latent space is robust due to the training of video diffusion models on large-scale web datasets, enabling the model to generalize well to out-of-domain scenes. As a result, LaLRM can efficiently reconstruct high-quality, large-scale, and generic 3D scenes.

Latent-based 3D Reconstruction Model For large-scale reconstruction, the authors draw inspiration from previous works and adopt a transformer-based architecture

that regresses Gaussian attributes in a pixel-aligned manner. The process begins with the video latent $z \in \mathbb{R}^{t \times h \times w \times c}$ and the corresponding camera poses $p \in \mathbb{R}^{T \times H \times W \times 6}$ in Plücker embedding format. These inputs are transformed into latent tokens and pose tokens. The video latent is patchified spatially, and the resulting tokens are used to represent the scene.

The Plücker embedding is patchified using 3D convolutions along both spatial and temporal dimensions to obtain pose tokens that match the length of the video latent tokens. These two token sets are then channel-wise concatenated, projected linearly to reduce the channel dimension, and processed through a series of transformer blocks.

To decode the video latent into a 3D reconstruction, a lightweight latent decoding module is employed. This module regresses Gaussian attributes by establishing pixel-level correspondences between the predicted Gaussians and the source video. The decoding involves a 3D-deconvolution layer with upsampling and results in a Gaussian feature map $G \in \mathbb{R}^{(T \cdot H \cdot W) \times 12}$, where the 12 channels correspond to different 3D Gaussian features such as RGB values, scale, rotation quaternion, opacity, and ray distance.

During training, the model renders images from the predicted Gaussians by selecting random supervision views, minimizing the reconstruction loss. The total loss function is a combination of mean squared error loss $\mathcal{L}_{\text{mse}}$ and perceptual loss $\mathcal{L}_{\text{perc}}$, with respective weight coefficients λ_1 and λ_2 , as defined in:

$$\mathcal{L}_{\text{recon}} = \lambda_1 \mathcal{L}_{\text{mse}} + \lambda_2 \mathcal{L}_{\text{perc}}$$

This combination ensures the model achieves high-quality 3D scene reconstructions.

ViewCrafter [5] employs a multi-step approach combining point cloud reconstruction, video diffusion models, and iterative view synthesis to achieve superior view synthesis results. Figure 2.5 illustrates the overall architecture of ViewCrafter.

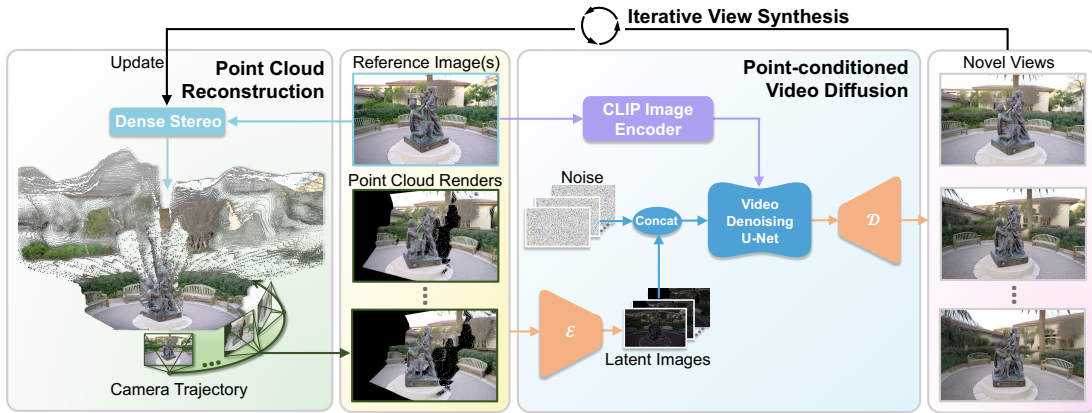


Figure 2.5: **Overview of ViewCrafter:** Given a reference image or sparse views, ViewCrafter constructs a point cloud using a dense stereo model for free-view rendering. A point-conditioned video diffusion model refines the coarse renders, addressing missing regions and distortions. Figure taken from the original paper [5].

Point Cloud Reconstruction in ViewCrafter To establish a structured 3D representation, ViewCrafter reconstructs a point cloud from the input images using a dense

stereo model, such as DUST3R [23]. This enables simultaneous point cloud generation and camera parameter estimation. The reconstruction assumes a centered principal point with square pixels, solving for focal length through an optimization procedure.

Given an input image $\mathbf{I}^0$, the model estimates point maps $\mathbf{O}^{0,0}$ and confidence maps $\mathbf{D}^{0,0}$, which inform the optimization process for recovering intrinsic camera parameters. This representation, however, suffers from missing regions and occlusions due to limited input views, necessitating further refinement.

High-Fidelity View Rendering in ViewCrafter To enhance the visual quality of the synthesized views, ViewCrafter employs a video diffusion model conditioned on the generated point cloud renders. The model is trained using latent diffusion methods, encoding both the reference images and the point cloud renders into a shared latent space.

During inference, the system renders a sequence of point cloud-based views, replacing the reference image render with the actual reference image. These representations are processed through a video denoising U-Net, which iteratively refines noisy latent representations into high-quality novel views.

Iterative View Synthesis and Camera Trajectory Planning in ViewCrafter A key limitation of existing view synthesis models is their struggle to generate long sequences while preserving stability. ViewCrafter addresses this by adopting an iterative approach, where new viewpoints are generated step-by-step to incrementally expand the point cloud and improve scene coverage.

To ensure optimal scene reconstruction, a Next-Best-View (NBV) strategy is employed to dynamically select subsequent camera poses. The system samples multiple candidate poses and evaluates them based on an adaptive utility function, selecting the pose that maximally reveals occluded regions and refines the synthesized scene.

Overall, ViewCrafter presents a robust approach to novel view synthesis, leveraging point cloud reconstruction, video diffusion-based refinement, and adaptive trajectory planning. By iteratively expanding the scene representation, it enables long-range, high-quality view synthesis suitable for applications in immersive rendering and 3D content generation. Our method is built upon the foundation of this work.

2.4 Summary

In this chapter, we reviewed key concepts and methodologies relevant to our work. We first discussed techniques for 3D reconstruction and novel view synthesis, highlighting the effectiveness of NeRF-based methods and 3D Gaussian Splatting. We then explored sparse-view 3D reconstruction, emphasizing Dust3R-based approaches for handling limited input views. Lastly, we examined the role of video diffusion models and depth priors in enhancing novel view synthesis, particularly in improving geometric consistency and visual quality.

This background establishes the foundation for our proposed method, situating it within the broader research landscape. In the next chapter, we will introduce our approach, detailing how we build upon these existing techniques to address the challenges of novel view synthesis.

Chapter 3

Methodology

3.1 Our Method

In our novel view synthesis framework, we initiate the process by constructing a point cloud representation from the given reference image(s) to achieve accurate pose control.

Generating Pointcloud from a single image

We use a dense stereo model like DUST3R [23] to simultaneously reconstruct the point cloud and estimate camera parameters, as opposed to depending on conventional Structure-from-Motion (SfM) and Multi-View Stereo (MVS) techniques, like those used in COLMAP [82], which require densely captured images for their effectiveness.

The Dust3R model processes a pair of RGB images, $\mathbf{I}^0, \mathbf{I}^1 \in \mathbb{R}^{H \times W \times 3}$, producing corresponding point maps $\mathbf{O}^{0,0}, \mathbf{O}^{1,0} \in \mathbb{R}^{H \times W \times 3}$ and confidence maps $\mathbf{D}^{0,0}, \mathbf{D}^{1,0} \in \mathbb{R}^{H \times W}$, which indicate the reliability of the generated points. The notation $\mathbf{O}^{0,0}, \mathbf{O}^{1,0}$ signifies that both maps are expressed in the coordinate system of the anchor view $\mathbf{I}^0$.

To recover the camera’s intrinsic parameters, we assume a centered principal point and square pixels, leaving the focal length f_0^* as the only unknown. This parameter is estimated through an optimization process using the Weiszfeld algorithm [83]:

$$f_0^* = f_0 \sum_{u=0}^W \sum_{v=0}^H \mathbf{D}_{u,v}^{0,0} \left\| (u', v') - f_0 \frac{(\mathbf{O}_{u,v,0}^{0,0}, \mathbf{O}_{u,v,1}^{0,0})}{\mathbf{O}_{u,v,2}^{0,0}} \right\|, \quad (3.1)$$

where $u' = u - \frac{W}{2}$ and $v' = v - \frac{H}{2}$.

For cases with a single input image, we generate a duplicate to create a paired input, allowing for the estimation of both the point map and camera intrinsics. When multiple input images are available, the approach extends to global alignment of point maps with additional optimization steps.

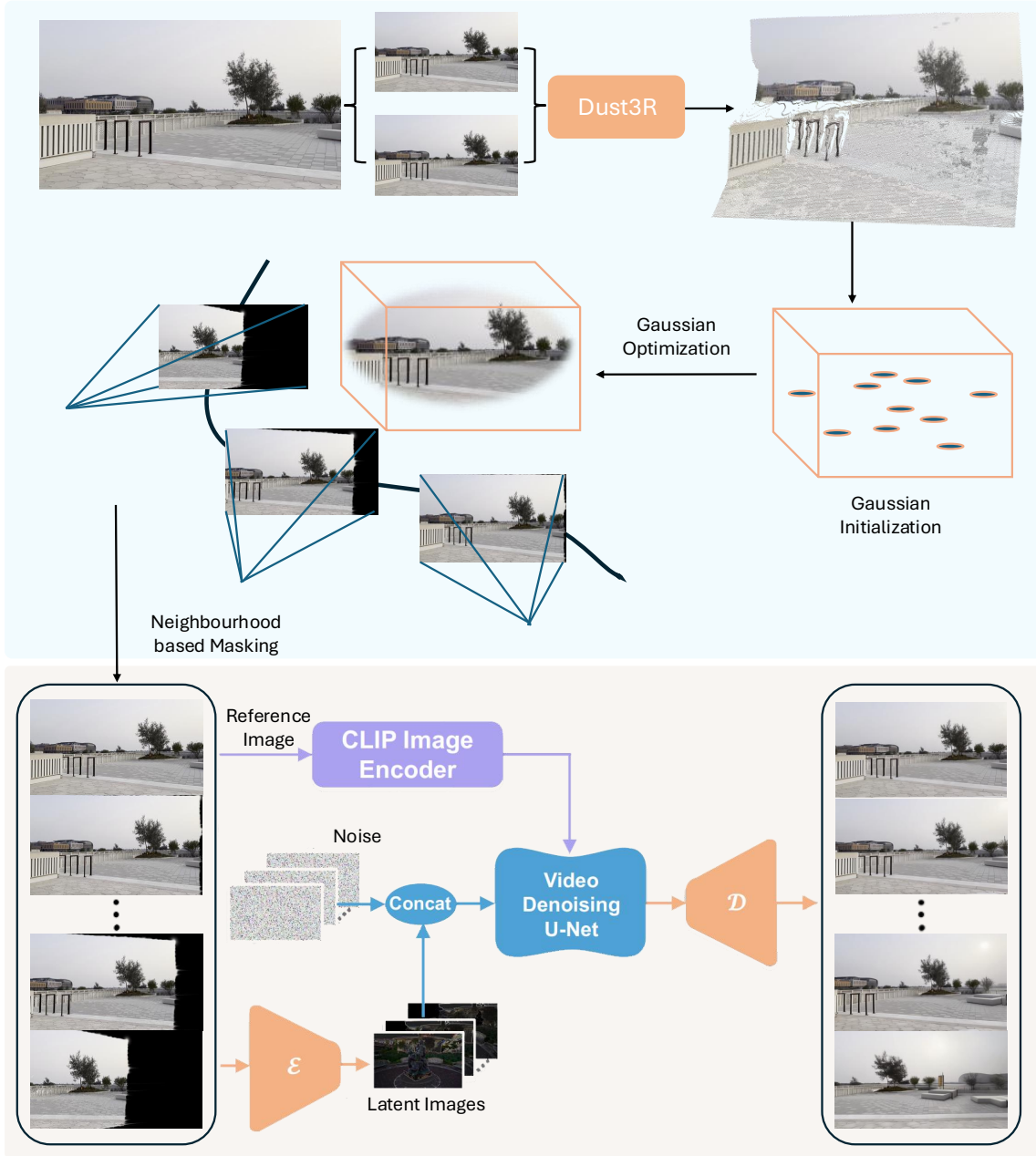


Figure 3.1: **Overall Architecture:** Given a single input image, we first duplicate it and process it through the Dust3R framework to obtain a point cloud representation. This point cloud is then used to initialize 3D Gaussians, which are subsequently optimized using a joint optimization strategy. Next, we apply differentiable rasterization to generate Gaussian renders, which are then passed into a pretrained video diffusion model for inpainting on the point cloud renders. To mitigate the distribution shift between point cloud renders and Gaussian splatting renders, we employ a neighborhood-based masking strategy before feeding the data into the diffusion model.

Generating Optimized Gaussian Splatting Renders

Then we utilize those point maps to generate 3D Gaussian splatting representations using a similar procedure to InstantSplat. This includes Adaptive Voxel Grid Partitioning, Confidence-aware Point Cloud Downsampling, and Joint Optimization for alignment.

Adaptive Voxel Grid Partitioning Given a set of N points in 3D space, we partition the domain into $K = k \times k \times k$ adaptive voxel grids. Each voxel V_j defines a spatial region, where the entire extent of the scene along each dimension (D) is uniformly divided into k segments. This results in a structured yet adaptive grid representation.

Each point $\mathbf{p}_i = (x_i, y_i, z_i)$ is assigned to a voxel based on its spatial coordinates, which fall within the computed voxel boundaries:

$$\begin{aligned} \text{Bound}_D &= \left\{ D_{\min} + \frac{n}{k}(D_{\max} - D_{\min}) \mid n = 0, 1, \dots, k-1 \right\} \\ V_j &= \{\mathbf{p}_i \mid \{x_i, y_i, z_i\} \in \{\text{Bound}_x, \text{Bound}_y, \text{Bound}_z\}\} \end{aligned}$$

This partitioning method ensures that points are efficiently grouped while maintaining a balanced distribution across the grid.

Confidence-aware Point Cloud Downsampling To reduce redundancy and improve computational efficiency, we downsample the point cloud within each voxel V_j . Instead of relying solely on confidence scores or performing global farthest point sampling (FPS), we apply FPS locally within each voxel in parallel. This approach ensures that selected points are spatially diverse, as FPS picks the farthest point from those already chosen in the sampling process.

A key observation is that raw point clouds often exhibit density biases due to overlapping regions in the training images. This imbalance can lead to an uneven representation, where confidence-based sampling alone may result in empty or underrepresented areas. By incorporating localized FPS, we achieve a more uniform distribution of points while preserving high-quality surface details. This refinement enhances the overall accuracy of the point cloud, making it more suitable for downstream processing.

Joint Optimization for Alignment To initialize Gaussians, we use the sampled point cloud following the heuristic rules proposed in 3D-GS [2]. However, noisy point clouds and inaccurate camera poses can introduce errors. To address this, we utilize a self-correction mechanism that iteratively refines both camera parameters and Gaussian properties through photometric loss optimization. This adaptive approach helps mitigate inconsistencies and improves the overall alignment of the reconstructed scene.

Gradient-Based Optimization for Self-Supervised 3D Scene Refinement In scenarios where multiple viewpoints and an initial coarse 3D representation with imprecise camera poses are available, modeled as a collection of Gaussians $\mathbf{G}$ and an initial transformation $\mathbf{T0}$, we employ gradient-based optimization to refine both the scene representation and camera parameters. This process aims to minimize the discrepancy between ground-truth pixel values and those rendered from the model. By simultaneously optimizing all

Gaussian parameters and refining the camera poses, $\mathbf{G}$ is adapted to achieve lower photometric errors when projected onto a target viewpoint $\mathbf{T}$. Mathematically, this optimization is expressed as:

$$\mathbf{G}^*, \mathbf{T}^* = \arg \min_{\mathbf{G}, \mathbf{T}} \sum_{v \in \mathcal{N}} \sum_{i=1}^{HW} \left\| \tilde{\mathbf{C}}_v^i(\mathbf{G}, \mathbf{T}) - \mathbf{C}_v^i(\mathbf{G}, \mathbf{T}) \right\| \quad (3.2)$$

This joint optimization ensures that the scene reconstruction aligns more accurately with the observed images, reducing photometric inconsistencies across different viewpoints. (See Subsection 4.4.)

However, the reconstructed point cloud may have missing regions, occlusions, and artefacts because of the constraints in point representation and the inadequate 3D information provided by sparse or single inputs, which could result in rendering results of lower quality. We use video diffusion models to overcome these obstacles, allowing for high-fidelity novel view synthesis even from defective point clouds.

A diffusion model [84] is fundamentally composed of two processes: a forward diffusion process, denoted as q , and a reverse denoising process, represented as p_θ . The forward process begins with an initial clean sample $x_0 \sim q_0(x_0)$ and gradually corrupts it by introducing Gaussian noise at each time step. This process is mathematically expressed as:

$$x_t = \alpha_t x_0 + \sigma_t \epsilon, \quad \epsilon \sim \mathcal{N}(0, \mathbf{I}), \quad (3.3)$$

where the parameters α_t and σ_t satisfy the constraint $\alpha_t^2 + \sigma_t^2 = 1$. The reverse process p_θ aims to recover the original data by removing noise using a trainable function ϵ_θ . The optimization objective for training this model is formulated as:

$$\min_{\theta} \mathbb{E}_{t \sim \mathcal{U}(0,1), \epsilon \sim \mathcal{N}(0, \mathbf{I})} \left[\left\| \epsilon_\theta(x_t, t) - \epsilon \right\|_2^2 \right]. \quad (3.4)$$

Latent Diffusion Models (LDMs) [85] are frequently used for diffusion-based video synthesis in order to improve computational efficiency. Rather of working in pixel space, LDMs use a pretrained Variational Autoencoder (VAE) to individually encode each frame of video data $x \in \mathbb{R}^{L \times 3 \times H \times W}$ into a latent representation:

$$z = \mathcal{E}(x), \quad z \in \mathbb{R}^{L \times C \times h \times w}. \quad (3.5)$$

The diffusion process, including both the forward transition q and the reverse denoising step p_θ , then operates within this latent space. Finally, the reconstructed video is obtained using the VAE decoder:

$$\hat{x} = \mathcal{D}(z). \quad (3.6)$$

Our foundation is based on DynamiCrafter [22], an open-source Image-to-Video (I2V) diffusion model that can produce motion sequences from a still image. This model is a logical choice for our methodology since it fits in nicely with our goal of synthesizing distinctive perspectives from sparse or limited input data.

However, ViewCrafter fine-tunes DynamiCrafter using point cloud renders. In contrast, when we directly replace these point cloud renders with Gaussian splatting outputs within the pipeline, we observe noticeable artifacts. These artifacts typically emerge near the boundaries where the existing rendered content meets the diffusion-generated regions. This issue arises due to inconsistencies between the high-fidelity Gaussian renderings and the generative nature of the diffusion model, which synthesizes missing details.

Neighborhood based Masking

To address the boundary inconsistencies observed in our results, we introduce a neighborhood-based masking strategy. This method selectively masks pixels based on the state of their neighboring pixels, ensuring a more seamless transition between the Gaussian-rendered content and the diffusion-generated regions.

Specifically, for each pixel, we examine its eight immediate neighbors. If at least five out of these eight neighboring pixels are already masked, the current pixel is also masked. This iterative masking process effectively smooths the blending region by progressively extending the mask into areas with high uncertainty, reducing sharp discontinuities at the boundary.

By enforcing this structured approach to masking, we achieve a more coherent integration of Gaussian-rendered content with the diffusion-based synthesis. This not only enhances the perceptual quality of the generated novel views but also preserves the structural integrity of the original renderings. (See Subsection 4.4)

In summary, the methodology presented in this thesis introduces a novel approach to view synthesis that combines point cloud generation, Gaussian splatting, and video diffusion models. We use the Dust3R framework for dense stereo point cloud reconstruction and apply joint optimization following Gaussian initialization to refine the splatting process, resulting in highly accurate 3D representations. The integration of video diffusion models, along with our neighborhood-based masking strategy, effectively addresses challenges such as boundary inconsistencies and missing regions in the rendered images. This approach enables high-fidelity novel view synthesis, even when working with sparse or imperfect input data, providing a robust solution for synthesizing views from limited image perspectives.

Chapter 4

Experiments and Results

4.1 Experimental Settings

To assess the view consistency of our approach, we employ multiple quantitative metrics that evaluate the fidelity, perceptual quality, and temporal coherence of the generated views.

Peak Signal-to-Noise Ratio (PSNR)

PSNR quantifies the ratio of the greatest achievable signal power to the power of corrupting noise, which indicates how faithful the created image is to the ground truth. It is described as:

$$\text{PSNR} = 10 \log_{10} \left(\frac{L^2}{\text{MSE}} \right), \quad (4.1)$$

where L is the maximum pixel value (e.g., 255 for 8-bit images), and MSE (Mean Squared Error) is given by:

$$\text{MSE} = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W (I_{ij} - \hat{I}_{ij})^2, \quad (4.2)$$

where I and $\hat{I}$ denote the ground truth and generated images, respectively.

Structural Similarity Index Measure (SSIM)

SSIM evaluates image quality by considering luminance, contrast, and structural information. It is computed as:

$$\text{SSIM}(I, \hat{I}) = \frac{(2\mu_I\mu_{\hat{I}} + C_1)(2\sigma_{I\hat{I}} + C_2)}{(\mu_I^2 + \mu_{\hat{I}}^2 + C_1)(\sigma_I^2 + \sigma_{\hat{I}}^2 + C_2)}, \quad (4.3)$$

where μ_I and $\mu_{\hat{I}}$ are the mean intensities, σ_I^2 and $\sigma_{\hat{I}}^2$ are the variances, and $\sigma_{I\hat{I}}$ is the covariance between I and $\hat{I}$. The constants C_1 and C_2 stabilize the division.

Learned Perceptual Image Patch Similarity (LPIPS)

LPIPS measures perceptual similarity by comparing deep feature representations extracted from a pre-trained neural network:

$$\text{LPIPS}(I, \hat{I}) = \sum_l w_l \|F_l(I) - F_l(\hat{I})\|_2^2, \quad (4.4)$$

where F_l denotes the activation maps at layer l and w_l are learned weights that emphasize perceptually important differences.

Fréchet Video Distance (FVD)

FVD evaluates temporal coherence and perceptual quality of generated videos by comparing feature distributions:

$$\text{FVD} = \|\mu_r - \mu_g\|^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}), \quad (4.5)$$

where (μ_r, Σ_r) and (μ_g, Σ_g) represent the mean and covariance of real and generated video feature distributions, respectively.

4.2 Datasets

For this work, we utilized a part of RealEstate10k test dataset and some random videos of both indoor and outdoor scenes taken from an iPhone 15 Pro Camera and their estimated camera parameters using COLMAP [86]. The RealEstate10K [87] dataset is a collection of 10,000 images sourced from real estate listings, designed for research in 3D reconstruction, localization, and image-based rendering. It includes high-quality images paired with corresponding camera poses, intrinsic parameters, and 3D scene information. The dataset's attributes include RGB images, camera intrinsics (3x3 matrices), transformation matrices (4x4), and 3D point cloud data. These images are taken from various angles and cover different parts of the property, offering diverse perspectives. The RealEstate10K dataset is widely used for tasks like visual localization, depth estimation, and creating 3D models from images.

Camera Intrinsics The camera intrinsic matrix, denoted as K , maps 3D coordinates in camera space to 2D image coordinates. It is represented as a 3×3 matrix:

$$K = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

Where:

- f_x and f_y are the focal lengths in the x and y directions (in pixels),
- s is the skew coefficient (usually zero),
- c_x and c_y are the optical center (principal point) coordinates.

This matrix transforms a 3D point in camera space, $[X_c \ Y_c \ Z_c]^T$, into 2D image coordinates $[u \ v \ 1]^T$ as follows:

$$[u \ v \ 1]^T = K \cdot [X_c \ Y_c \ Z_c]^T$$

Transformation Matrix The transformation matrix T converts points from world coordinates to camera coordinates. It is a 4×4 matrix that includes both rotation and translation:

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

Where:

- R is the 3×3 rotation matrix,
- t is the 3×1 translation vector.

To transform a point $[X_w \ Y_w \ Z_w]^T$ in world coordinates into $[X_c \ Y_c \ Z_c]^T$ in camera coordinates:

$$[X_c \ Y_c \ Z_c \ 1]^T = T \cdot [X_w \ Y_w \ Z_w \ 1]^T$$

To project a 3D point in world coordinates into 2D image coordinates, we combine the intrinsic matrix K and the transformation matrix T :

$$[u \ v \ 1]^T = K \cdot T \cdot [X_w \ Y_w \ Z_w \ 1]^T$$

4.3 Implementation Details

We conducted our experiments on a single NVIDIA A6000 GPU with 48GB of memory. Our inference process is performed at a resolution of 1024×576 , with DDIM sampling steps set to 50. We sample our evaluation data for 25 frames and use COLMAP to generate the camera trajectory, which we consider as the ground-truth camera trajectory. Each generated video consists of 25 frames and the inference process for a single image takes approximately 2 minutes.

4.4 Results

Importance of utilizing 3D Gaussian Splatting

Our study introduces a method that combines the generative capabilities of video diffusion models with the structural guidance of Gaussian splatting-based renders, instead of relying on point cloud-based renders.

Figure 4.1 shows the improvement in the use of optimized Gaussian splatting renderings compared to point-cloud-based renderings.



Figure 4.1: Improvement from 3DGS-Rendering Compared to PointCloud Renders; There’s a noticeable improvement within the bounding boxes shown in red.

Importance of utilizing Neighborhood based Masking

Our neighborhood-based masking strategy that selectively masks pixels based on their neighbors, ensuring smoother transitions between Gaussian-rendered content and diffusion-generated regions.

Figure 4.2 shows the improvement of the results with and without the masking strategy.

In addition, we present both qualitative and quantitative results for our method in different experimental settings.



Figure 4.2: Comparison of Results with and without Neighborhood based Masking; There is a clear improvement around the red bounding box showcasing the importance of the masking strategy.

Quantitative Results

We evaluated our method in both indoor and outdoor scenes. Table 4.1 presents the quantitative results for outdoor scenes. The results are based on 10 different scenes and each scene with 25 views. Our method significantly outperforms the baseline in FVD, PSNR, and LPIPS while achieving comparable performance in SSIM.

Method	FVD ↓	SSIM ↑	PSNR ↑	LPIPS ↓
ViewCrafter	158.828	0.617	18.78	0.1840
Ours	147.064	0.612	19.03	0.1691

Table 4.1: Quantitative comparison of different methods for outdoor data. The best values are highlighted in **bold**. Lower FVD and LPIPS values are better, while higher SSIM and PSNR values indicate improved performance.

Similarly, Table 4.2 shows the performance of our method over indoor scenes.

Method	FVD ↓	SSIM ↑	PSNR ↑	LPIPS ↓
ViewCrafter	163.904	0.628	19.03	0.1361
Ours	176.116	0.632	19.05	0.1279

Table 4.2: Quantitative comparison of different methods for indoor data. The best values are highlighted in **bold**. Lower FVD and LPIPS values are better, while higher SSIM and PSNR values indicate improved performance.

Our evaluation across both indoor and outdoor scenes demonstrates the effectiveness of our method in comparison to the baseline. In outdoor scenes, our approach achieves significantly better FVD, PSNR, and LPIPS scores while maintaining comparable SSIM performance. For indoor scenes, our method attains the best PSNR and LPIPS scores,

along with a slight improvement in SSIM, though with a higher FVD. These results indicate that our method consistently enhances perceptual quality and reconstruction fidelity while maintaining competitive structural similarity. Overall, our approach proves to be robust across diverse environments, effectively improving visual quality and coherence.

Further, we also evaluate our method on RealEstate-10K[87] data. We consider 10 scenes in the RealEstate-10K test set and evaluate our method on it and consider sampled 25 frames from each video. Table 4.3 shows the quantitative results for these data sets.

Method	FVD ↓	SSIM ↑	PSNR ↑	LPIPS ↓
ViewCrafter	215.187	0.694	22.21	0.0895
Ours	211.876	0.694	22.39	0.1024

Table 4.3: Quantitative comparison of different methods for RealEstate-10K data. The best values are highlighted in **bold**. Lower FVD and LPIPS values are better, while higher SSIM and PSNR values indicate improved performance.

Our evaluation on the RealEstate-10K dataset further confirms the effectiveness of our approach, achieving the best FVD and PSNR scores while keeping SSIM and LPIPS competitive. These results demonstrate the robustness and versatility of our method in handling complex real-world scenes, ensuring high-quality visual synthesis with improved coherence across different environments.

Qualitative Results

We evaluate our method qualitatively on both indoor and outdoor datasets. Figure 4.3 presents a comparative analysis of our method’s performance on outdoor data. For benchmarking, we use ViewCrafter [5] as our baseline framework and compare our results against it.

Figure 4.4 showcase the comparative performance of our method for indoor scenes.

The results demonstrate that our method exceeds the existing approach by improving the quality of the rendering, ensuring better view consistency, and capturing fine details more effectively. Additionally, our approach improves smooth interpolation, reduces artifacts, and better preserves view-dependent effects compared to baseline.

Other Experiments

Iterating through the diffusion model of ViewCrafter

One of our initial experiments was to investigate the effect of iterating through the diffusion model of ViewCrafter multiple times. However, as shown in Figure 4.5, we observed that additional iterations did not lead to any significant improvement in the generated results.



Figure 4.3: **Qualitative Comparison of Outdoor Scenes:** Results shown are sampled frames from a 25-frame sequence. Frame #1 represents the input reference image, while the subsequent frames illustrate the synthesized novel views. Our method demonstrates superior rendering quality, improved view consistency, and better preservation of fine details compared to the baseline.



Figure 4.4: **Qualitative Comparison of Indoor Scenes:** Sampled frames from a 25-frame sequence are shown, with Frame #1 representing the input reference image. The subsequent frames depict the synthesized novel views, highlighting the improved rendering quality, enhanced view consistency, and better preservation of fine details achieved by our method compared to the baseline.



Figure 4.5: Visualization of iterative processing through the diffusion model of ViewCrafter. The first row represents the output after the first iteration, while the second row shows the result after the second iteration.

Generate Gaussian Splatting Renders from diffusion output

We identified that the results generated by the video diffusion model are inconsistent between frames. To address this, we used the output from the video diffusion model to fit Gaussian splatting and then rendered the same camera trajectory. This approach aims to produce more view-consistent outputs compared to our baseline method.

However, we observed that, due to inconsistencies and distribution shifts in the data, the output contains numerous artifacts. Consequently, if we attempt to iterate these artifact-laden results through Gaussian splatting within the diffusion model, it will not correct them, as the diffusion model is not trained for this purpose.

Utilizing depth priors

Due to artifacts generated from Gaussian splatting from diffusion-generated images, we incorporated depth priors and regress Gaussian splatting directly similar to the DepthSplat [3] paper. The figure 4.6 shows the results on the generated gaussian splatting renders.

However, the results clearly indicate that DepthSplat [3] struggles to recover accurate depth information from distorted diffusion-generated images. This suggests that our approach cannot be effectively extended in this direction while keeping DepthSplat frozen for generating novel views from diffusion outputs.

Intermediate Test time Refinement

In high-fidelity image synthesis and denoising tasks, maintaining temporal and structural consistency is crucial. A common challenge in denoising models is ensuring smooth and stable progression across intermediate steps, preventing inconsistencies that may arise from stochastic variations in the denoising process. We propose an approach that integrates intermediate denoising supervision with Gaussian splatting, leveraging additional structural cues to enhance overall consistency.

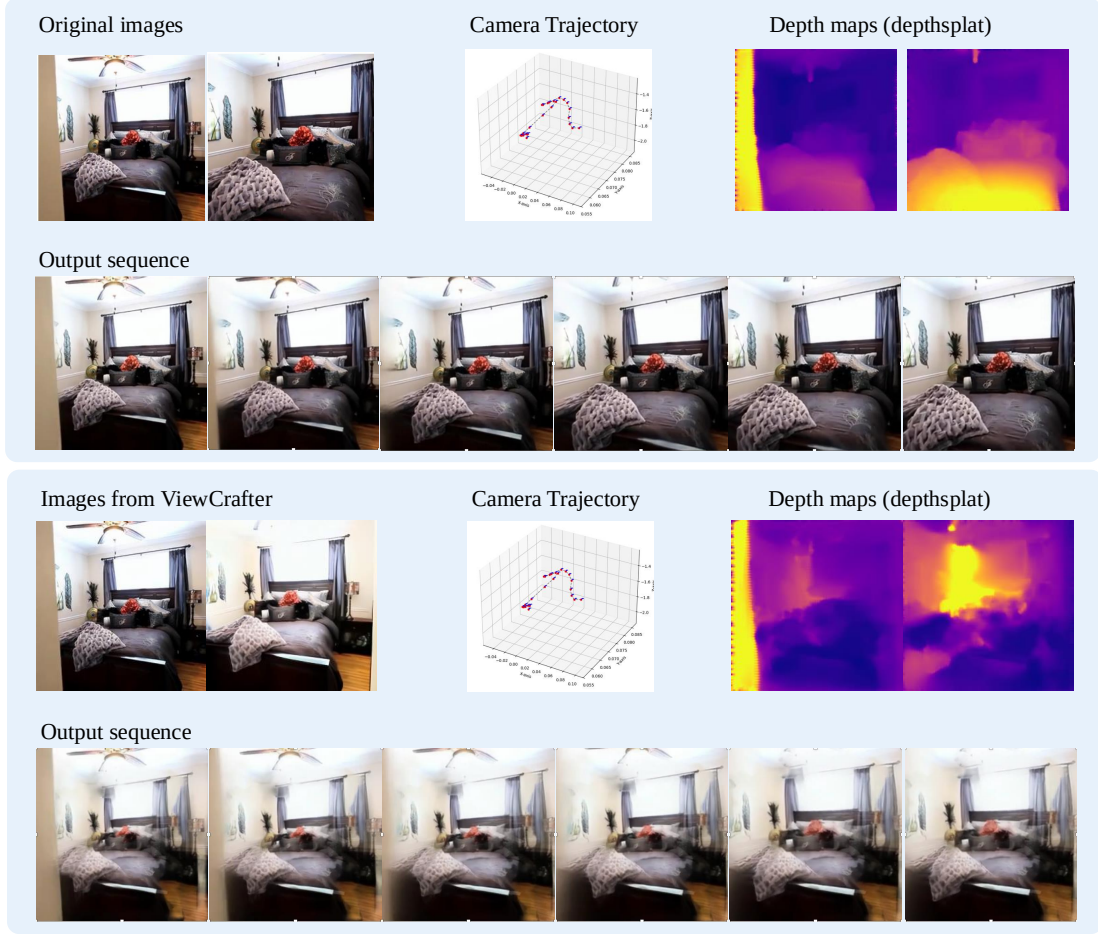


Figure 4.6: Results of Failure by Incooperating Depthmaps on Diffusion Generated Inconsistent Views; Results showcase that utilizing diffusion generated images for depth regression has significantly degrade the performance.

At test time, instead of following a standard denoising trajectory, we extract intermediate outputs from the denoising process and utilize them to generate a Gaussian splatting representation. See figure 4.7. This representation is then fed back into the remaining denoising steps, allowing the model to refine its predictions with an additional structural prior.

Gaussian splatting, a technique originally used for efficient 3D reconstruction, represents a scene or object as a collection of anisotropic Gaussian kernels in a continuous space. By incorporating this representation into the denoising process, we enforce spatial coherence and prevent the model from diverging from meaningful structures as the denoising steps progress.

However, as shown in Figure 4.8, while incorporating Gaussian splatting at later denoising stages improves the results, it also introduces additional artifacts. Since the diffusion model is not explicitly trained to remove these artifacts, they persist in the final output, highlighting a limitation of this approach.

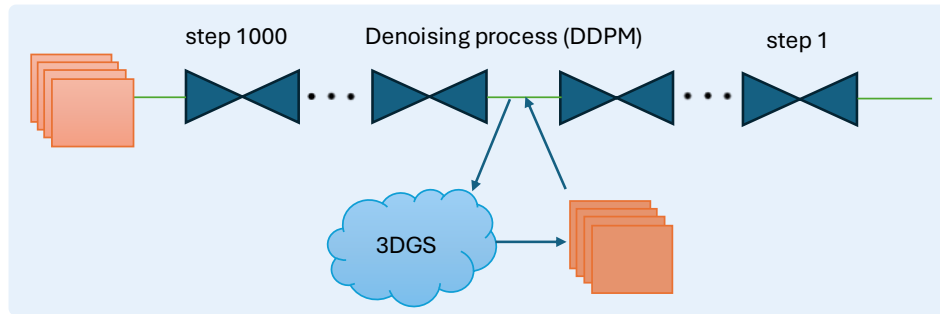


Figure 4.7: Intermediate supervision: At denoising process we take the output at a middle step of the process and generate gaussian splatting renders from those and pass it back to the denoising process

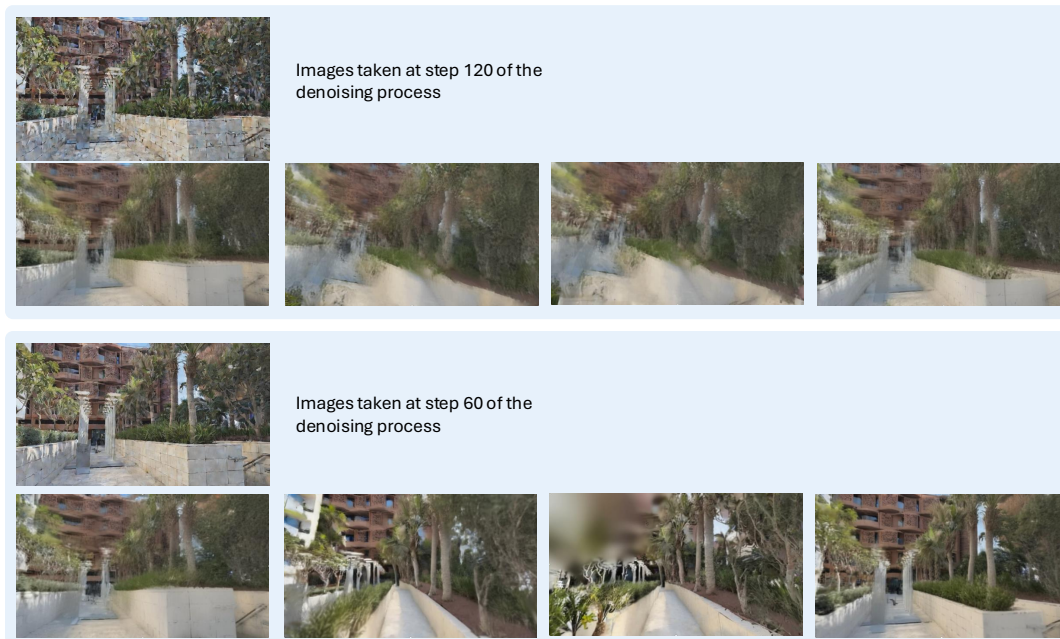


Figure 4.8: Denoising results with intermediate Gaussian splatting supervision. When intermediate supervision is conducted at later denoising steps, it leads to better results, however, pretrained diffusion model fails to remove the artifacts generated from Gaussian Splatting.

Chapter 5

Conclusion

5.1 Summary

In this work, we study a new approach for single-image 3D reconstruction that leverages the strengths of video diffusion models and Gaussian splatting to achieve high-quality novel view synthesis while ensuring geometric and photometric consistency across view-points. Traditional methods, such as regression-based models and point cloud rendering techniques, often suffer from limited generalization, sparsity artifacts, and inconsistencies in occluded regions. Meanwhile, diffusion-based approaches, though powerful, struggle with precise camera control and structural coherence. Our proposed framework addresses these limitations by incorporating explicit 3D information derived from Gaussian splatting, enabling smooth interpolation, view-dependent effects, and photorealistic rendering.

Our experimental results demonstrate that our method outperforms existing approaches in terms of rendering quality, consistency, and controllability of the camera pose. By conditioning video diffusion priors on structured 3D representations, our approach effectively captures fine details, maintains view consistency, and mitigates common issues such as missing geometry and distortions in sparse-view scenarios. Furthermore, Gaussian splatting provides a more robust rendering pipeline compared to point cloud-based methods, offering superior handling of transparency, soft edges, and occlusions.

5.2 Limitations and Future Directions

Despite its advantages, our method has certain limitations. Since we utilize a pretrained diffusion model originally trained on point-cloud renders, directly incorporating Gaussian splatting-based renders into the diffusion process introduces a distribution shift. Although we employed a neighborhood-based masking strategy to mitigate the resulting artifacts, fine-tuning the diffusion model on Gaussian-rendered data could further enhance perfor-

mance. Additionally, our approach relies on explicit 3D guidance, which may not generalize well to highly complex or unstructured scenes without further adaptations.

For future work, we aim to enhance the diffusion model by training it specifically on Gaussian splatting renders, reducing inconsistencies, and improving overall synthesis quality. Another promising direction is extending our approach to dynamic scenes, where ensuring temporal coherence is essential for applications such as video synthesis and augmented reality. Additionally, optimizing Gaussian splatting for real-time rendering could expand its applicability to interactive experiences. Finally, increasing trajectory coverage, incorporating longer trajectories, and ensuring consistency in dynamic scenes are key areas for further exploration.

By addressing these challenges, we aim to further improve the robustness and generalization of our method, making it a more practical solution for single-image 3D reconstruction and novel view synthesis.

References

- [1] Z. Wang, S. Wu, W. Xie, M. Chen, and V. A. Prisacariu, “Nerf-: Neural radiance fields without known camera parameters,” *arXiv preprint arXiv:2102.07064*, 2021.
- [2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, “3d gaussian splatting for real-time radiance field rendering,” *ACM TOG*, 2023.
- [3] H. Xu, S. Peng, F. Wang, H. Blum, D. Barath, A. Geiger, and M. Pollefeys, “Depth-splat: Connecting gaussian splatting and depth,” *arXiv preprint arXiv:2410.13862*, 2024.
- [4] H. Liang, J. Cao, V. Goel, G. Qian, S. Korolev, D. Terzopoulos, K. N. Plataniotis, S. Tulyakov, and J. Ren, “Wonderland: Navigating 3d scenes from a single image,” *arXiv preprint arXiv:2412.12091*, 2024.
- [5] W. Yu, J. Xing, L. Yuan, W. Hu, X. Li, Z. Huang, X. Gao, T.-T. Wong, Y. Shan, and Y. Tian, “Viewcrafter: Taming video diffusion models for high-fidelity novel view synthesis,” *arXiv preprint arXiv:2409.02048*, 2024.
- [6] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, “Nerf: Representing scenes as neural radiance fields for view synthesis,” in *ECCV*, 2020.
- [7] O. Wiles, G. Gkioxari, R. Szeliski, and J. Johnson, “Synsin: End-to-end view synthesis from a single image,” in *CVPR*, 2020.
- [8] R. Rombach, P. Esser, and B. Ommer, “Geometry-free view synthesis: Transformers and no 3d priors,” in *ICCV*, 2021.
- [9] C. Rockwell, D. F. Fouhey, and J. Johnson, “Pixelsynth: Generating a 3d-consistent experience from a single image,” in *ICCV*, 2021.
- [10] B. Park, H. Go, and C. Kim, “Bridging implicit and explicit geometric transformation for single-image view synthesis,” *IEEE TPAMI*, 2024.

- [11] T. Zhou, R. Tucker, J. Flynn, G. Fyffe, and N. Snavely, “Stereo magnification: Learning view synthesis using multiplane images,” *ACM TOG*, 2018.
- [12] Y. Han, R. Wang, and J. Yang, “Single-view view synthesis in the wild with learned adaptive multiplane images,” in *SIGGRAPH Conference*, 2022.
- [13] R. Tucker and N. Snavely, “Single-view view synthesis with multiplane images,” in *CVPR*, 2020.
- [14] A. Yu, V. Ye, M. Tancik, and A. Kanazawa, “pixelnerf: Neural radiance fields from one or few images,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 4578–4587.
- [15] R. Liu, R. Wu, B. Van Hoorick, P. Tokmakov, S. Zakharov, and C. Vondrick, “Zero-1-to-3: Zero-shot one image to 3d object,” in *ICCV*, 2023.
- [16] K. Sargent, Z. Li, T. Shah, C. Herrmann, H.-X. Yu, Y. Zhang, E. R. Chan, D. Lagun, L. Fei-Fei, D. Sun, and J. Wu, “ZeroNVS: Zero-shot 360-degree view synthesis from a single real image,” in *CVPR*, 2024.
- [17] Z. Wang, Z. Yuan, X. Wang, T. Chen, M. Xia, P. Luo, and Y. Shan, “Motionctrl: A unified and flexible motion controller for video generation,” in *SIGGRAPH Conference*, 2024.
- [18] J. Chung, S. Lee, H. Nam, J. Lee, and K. M. Lee, “Luciddreamer: Domain-free generation of 3d gaussian splatting scenes,” *arXiv preprint arXiv:2311.13384*, 2023.
- [19] J. Shriram, A. Trevithick, L. Liu, and R. Ramamoorthi, “Realmdreamer: Text-driven 3d scene generation with inpainting and depth diffusion,” *arXiv preprint arXiv:2404.07199*, 2024.
- [20] P.-Y. Lab and T. A. etc., “Open-sora-plan,” <https://github.com/PKU-YuanGroup/Open-Sora-Plan>, 2024. [Online]. Available: <https://doi.org/10.5281/zenodo.10948109>
- [21] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, A. Letts *et al.*, “Stable video diffusion: Scaling latent video diffusion models to large datasets,” *arXiv preprint arXiv:2311.15127*, 2023.
- [22] J. Xing, M. Xia, Y. Zhang, H. Chen, X. Wang, T.-T. Wong, and Y. Shan, “Dyna-icrafter: Animating open-domain images with video diffusion priors,” *arXiv preprint arXiv:2310.12190*, 2023.
- [23] S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii, and J. Revaud, “Dust3r: Geometric 3d vision made easy,” in *CVPR*, 2024.
- [24] V. Leroy, Y. Cabon, and J. Revaud, “Grounding image matching in 3d with mast3r,” *arXiv:2406.09756*, 2024.
- [25] J.-M. F. Johannes Lutz Schönberger, “Structure-from-motion revisited,” in *CVPR*, 2016.

- [26] H. Bay, T. Tuytelaars, and L. V. Gool, “Surf: Speeded up robust features,” in *ECCV*. Springer, 2006, pp. 404–417.
- [27] E. Rosten and T. Drummond, “Machine learning for high-speed corner detection,” in *ECCV*. Springer, 2006.
- [28] A. Barroso-Laguna, E. Riba, D. Ponsa, and K. Mikolajczyk, “Key. net: Keypoint detection by handcrafted and learned cnn filters,” in *ICCV*, 2019, pp. 5836–5844.
- [29] J. Revaud, C. R. de Souza, M. Humenberger, and P. Weinzaepfel, “R2D2: reliable and repeatable detector and descriptor,” pp. 12 405–12 415, 2019.
- [30] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, “Superglue: Learning feature matching with graph neural networks,” in *CVPR*, 2020, pp. 4937–4946.
- [31] D. Barath, D. Mishkin, L. Cavalli, P.-E. Sarlin, P. Hruby, and M. Pollefeys, “Affineglue: Joint matching and robust estimation,” 2023.
- [32] P. Lindenberger, P.-E. Sarlin, V. Larsson, and M. Pollefeys, “Pixel-perfect structure-from-motion with featuremetric refinement,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 5987–5997.
- [33] Y. Xiao, N. Xue, T. Wu, and G.-S. Xia, “Level-set fm: Structure from motion on neural level set of implicit surfaces,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 17 205–17 214.
- [34] H. A. s, R. R. l Jensen, G. Vogiatzis, E. Tola, and A. B. Dahl, “Large-scale data for multiple-view stereopsis,” *IJCV*, 2016.
- [35] C. Yeshwanth, Y.-C. Liu, M. Nießner, and A. Dai, “Scannet++: A high-fidelity dataset of 3d indoor scenes,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 12–22.
- [36] Y. Furukawa, C. Hernández *et al.*, “Multi-view stereo: A tutorial,” *Foundations and trends® in Computer Graphics and Vision*, vol. 9, no. 1-2, pp. 1–148, 2015.
- [37] J. Zhang, S. Li, Z. Luo, T. Fang, and Y. Yao, “Vis-mvsnet: Visibility-aware multi-view stereo network,” *International Journal of Computer Vision*, vol. 131, no. 1, pp. 199–214, 2023.
- [38] J.-W. Bian, H. Zhan, N. Wang, T.-J. Chin, C. Shen, and I. D. Reid, “Auto-rectify network for unsupervised indoor depth estimation,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 12, pp. 9802–9813, 2022.
- [39] W. Yin, J. Zhang, O. Wang, S. Niklaus, S. Chen, Y. Liu, and C. Shen, “Towards accurate reconstruction of 3d scene shape from a single monocular image,” *IEEE TPAMI*, 2022.
- [40] E. Brachmann, A. Krull, S. Nowozin, J. Shotton, F. Michel, S. Gumhold, and C. Rother, “Dsac - differentiable ransac for camera localization,” in *CVPR*, 2017.

- [41] E. Brachmann and C. Rother, “Learning less is more - 6d camera localization via 3d surface regression,” in *CVPR*, 2018.
- [42] —, “Visual camera re-localization from rgb and rgb-d images using dsac,” *PAMI*, 2022.
- [43] E. Penner and L. Zhang, “Soft 3d reconstruction for view synthesis,” *ACM Transactions on Graphics (TOG)*, vol. 36, no. 6, pp. 1–11, 2017.
- [44] V. Sitzmann, J. Thies, F. Heide, M. Nießner, G. Wetzstein, and M. Zollhofer, “Deepvoxels: Learning persistent 3d feature embeddings,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 2437–2446.
- [45] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, “Mip-nerf 360: Unbounded anti-aliased neural radiance fields,” in *CVPR*, 2022.
- [46] A. Yu, R. Li, M. Tancik, H. Li, R. Ng, and A. Kanazawa, “Plenotrees for real-time rendering of neural radiance fields,” in *ICCV*, 2021.
- [47] S. J. Garbin, M. Kowalski, M. Johnson, J. Shotton, and J. Valentin, “Fastnerf: High-fidelity neural rendering at 200fps,” in *ICCV*, 2021.
- [48] C. Sun, M. Sun, and H.-T. Chen, “Direct voxel grid optimization: Super-fast convergence for radiance fields reconstruction,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 5459–5469.
- [49] T. Müller, A. Evans, C. Schied, and A. Keller, “Instant neural graphics primitives with a multiresolution hash encoding,” *ACM Transactions on Graphics (ToG)*, vol. 41, no. 4, pp. 1–15, 2022.
- [50] S. Fridovich-Keil, A. Yu, M. Tancik, Q. Chen, B. Recht, and A. Kanazawa, “Plenoxels: Radiance fields without neural networks,” in *CVPR*, 2022.
- [51] G. Kopanas, J. Philip, T. Leimkühler, and G. Drettakis, “Point-based neural rendering with per-view optimization,” in *Computer Graphics Forum*, vol. 40, no. 4. Wiley Online Library, 2021, pp. 29–43.
- [52] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “Pytorch: An imperative style, high-performance deep learning library,” *Advances in neural information processing systems*, vol. 32, 2019.
- [53] P. Weinzaepfel, V. Leroy, T. Lucas, R. Brégier, Y. Cabon, V. Arora, L. Antsfeld, B. Chidlovskii, G. Csurka, and J. Revaud, “Croco: Self-supervised pre-training for 3d vision tasks by cross-view completion,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 3502–3516, 2022.
- [54] P. Weinzaepfel, T. Lucas, V. Leroy, Y. Cabon, V. Arora, R. Brégier, G. Csurka, L. Antsfeld, B. Chidlovskii, and J. Revaud, “Croco v2: Improved cross-view completion pre-training for stereo matching and optical flow,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 17 969–17 980.

- [55] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
- [56] A. v. d. Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.
- [57] Z. Fan, W. Cong, K. Wen, K. Wang, J. Zhang, X. Ding, D. Xu, B. Ivanovic, M. Pavone, G. Pavlakos *et al.*, “Instantsplat: Unbounded sparse-view pose-free gaussian splatting in 40 seconds,” *arXiv:2403.20309*, 2024.
- [58] Y. Yao, Z. Luo, S. Li, T. Fang, and L. Quan, “Mvsnet: Depth inference for unstructured multi-view stereo,” in *ECCV*, 2018.
- [59] S. F. Bhat, I. Alhashim, and P. Wonka, “Zoedepth: Zero-shot transfer by combining relative and metric depth,” 2023.
- [60] W. Yin, C. Zhang, H. Chen, Z. Cai, G. Yu, K. Wang, X. Chen, and C. Shen, “Metric3d: Towards zero-shot metric 3d prediction from a single image,” in *CVPR*, 2023.
- [61] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, “Depth anything: Unleashing the power of large-scale unlabeled data,” in *CVPR*, 2024, natexlab.
- [62] L. Yang, B. Kang, Z. Huang, Z. Zhao, X. Xu, J. Feng, and H. Zhao, “Depth anything v2,” 2024, natexlab.
- [63] D. Charatan, S. Li, A. Tagliasacchi, and V. Sitzmann, “pixelsplat: 3d gaussian splats from image pairs for scalable generalizable 3d reconstruction,” 2024.
- [64] Y. Chen, H. Xu, C. Zheng, B. Zhuang, M. Pollefeys, A. Geiger, T.-J. Cham, and J. Cai, “Mvsplat: Efficient 3d gaussian splatting from sparse multi-view images,” 2024.
- [65] Y. Xu, Z. Shi, W. Yifan, H. Chen, C. Yang, S. Peng, Y. Shen, and G. Wetzstein, “Grm: Large gaussian reconstruction model for efficient 3d reconstruction and generation,” 2024, natexlab.
- [66] K. Zhang, S. Bi, H. Tan, Y. Xiangli, N. Zhao, K. Sunkavalli, and Z. Xu, “Gs-lrm: Large reconstruction model for 3d gaussian splatting,” 2024, natexlab.
- [67] S. Szymanowicz, C. Rupprecht, and A. Vedaldi, “Splatter image: Ultra-fast single-view 3d reconstruction,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 10 208–10 217.
- [68] S. Szymanowicz, E. Insafutdinov, C. Zheng, D. Campbell, J. F. Henriques, C. Rupprecht, and A. Vedaldi, “Flash3d: Feed-forward generalisable 3d scene reconstruction from a single image,” *arXiv preprint arXiv:2406.04343*, 2024.

- [69] C. Zhang, Y. Zou, Z. Li, M. Yi, and H. Wang, “Transplat: Generalizable 3d gaussian splatting from sparse multi-view images with transformers,” *arXiv preprint arXiv:2408.13770*, 2024, natexlaba.
- [70] M. Turkulainen, X. Ren, I. Melekhov, O. Seiskari, E. Rahtu, and J. Kannala, “Dn-splatter: Depth and normal priors for gaussian splatting and meshing,” *arXiv preprint arXiv:2403.17822*, 2024.
- [71] Y. Liu, K. Zhang, Y. Li, Z. Yan, C. Gao, R. Chen, Z. Yuan, Y. Huang, H. Sun, J. Gao *et al.*, “Sora: A review on background, technology, limitations, and opportunities of large vision models,” *arXiv preprint arXiv:2402.17177*, 2024.
- [72] A. Blattmann, T. Dockhorn, S. Kulal, D. Mendelevitch, M. Kilian, D. Lorenz, Y. Levi, Z. English, V. Voleti, and A. Letts, “Stable video diffusion: Scaling latent video diffusion models to large datasets,” 2023.
- [73] W. Peebles and S. Xie, “Scalable diffusion models with transformers,” in *ICCV*, 2023.
- [74] S. Bahmani, I. Skorokhodov, A. Siarohin, W. Menapace, G. Qian, M. Vasilkovsky, H.-Y. Lee, C. Wang, J. Zou, A. Tagliasacchi *et al.*, “Vd3d: Taming large video diffusion transformers for 3d camera control,” *arXiv preprint arXiv:2407.12781*, 2024.
- [75] Z. Wang, Z. Yuan, X. Wang, Y. Li, T. Chen, M. Xia, P. Luo, and Y. Shan, “Motionctrl: A unified and flexible motion controller for video generation,” in *ACM SIGGRAPH 2024 Conference Papers*, 2024, pp. 1–11.
- [76] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” in *ICLR*, 2022.
- [77] D. Xu, W. Nie, C. Liu, S. Liu, J. Kautz, Z. Wang, and A. Vahdat, “Camco: Camera-controllable 3d-consistent image-to-video generation,” *arXiv preprint arXiv:2406.02509*, 2024.
- [78] A. Chen, Z. Xu, F. Zhao, X. Zhang, F. Xiang, J. Yu, and H. Su, “Mvsnerf: Fast generalizable radiance field reconstruction from multi-view stereo,” in *ICCV*, 2021.
- [79] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin transformer: Hierarchical vision transformer using shifted windows,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2021, pp. 10 012–10 022.
- [80] H. Xu, J. Zhang, J. Cai, H. Rezatofighi, F. Yu, D. Tao, and A. Geiger, “Unifying flow, stereo and depth estimation,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 11, pp. 13 941–13 958, 2023.
- [81] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, “High-resolution image synthesis with latent diffusion models,” in *CVPR*, 2022.
- [82] J. L. Schönberger and J.-M. Frahm, “Structure-from-motion revisited,” in *CVPR*, 2016.

- [83] F. Plastria, “The weiszfeld algorithm: proof, amendments and extensions, ha eiselt and v. marianov (eds.) foundations of location analysis, international series in operations research and management science,” 2011.
- [84] J. Song, C. Meng, and S. Ermon, “Denoising diffusion implicit models,” in *ICLR*, 2021.
- [85] G. Metzger, E. Richardson, O. Patashnik, R. Giryes, and D. Cohen-Or, “Latent-nerf for shape-guided generation of 3d shapes and textures,” in *CVPR*, 2023.
- [86] J. L. Schönberger, E. Zheng, J.-M. Frahm, and M. Pollefeys, “Pixelwise view selection for unstructured multi-view stereo,” in *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part III 14*. Springer, 2016, pp. 501–518.
- [87] T. Zhou, R. Tucker, J. Flynn, G. Fyffe, and N. Snavely, “Stereo magnification: Learning view synthesis using multiplane images,” *ACM TOG*, 2018.